

LOGIC CIRCUIT LAB Report

4조

김현우 2020311081

유지훈 2022314742

이승우 2022314820

박진우 2021311410

2025 2nd semester



성균관대학교
전자전기공학부

1. Objective of the Experiment

Convolutional Neural Networks(CNNs)는 이미지 인식, 자기주도 학습, 의료 영상 처리 등 다양한 인공지능 분야에서 핵심적 역할을 수행한다. 그러나 Convolution 연산은 대규모 Multiply-Accumulate(MAC) 연산과 빈번한 메모리 접근을 요구하는 고연산·고대역폭 구조이며, 이러한 특성은 범용 CPU·GPU에서는 메모리 병목과 연산 지연을 야기하는 주요 원인으로 작용한다.

특히 CNN에서 발생하는 연산량 대비 데이터 이동 비용(data movement cost)은 전체 에너지 소모의 상당 부분을 차지하며, 효율적인 하드웨어 가속을 위해서는 연산 자원보다 데이터 재사용(Data Reuse)과 지역적 통신(Local Communication)을 중심으로 한 구조 설계가 필수적이다.

Systolic Array는 이러한 요구를 충족하기 위한 대표적인 CNN 가속 구조로, 입력 feature map과 weight가 규칙적인 방향으로 이동하며 PE 간 국소적으로 전달되는 특성을 이용해 메모리 접근 최소화, 파이프라인 유지, 연산 효율 극대화를 달성한다.

본 실험은 이러한 Systolic Array 구조를 FPGA 기반으로 설계·구현하고, 그 구조적 차이에 따른 성능 변화를 실험적으로 검증하는 것을 목표로 한다.

본 프로젝트의 목표는 아래와 같다.

(1) 2D Convolution의 하드웨어적 연산 모델 분석

- Sliding Window 기반 Convolution 연산을 데이터플로우(dataflow) 중심의 하드웨어 관점에서 재정의한다.
- ifmap 이동 방향, weight 유지/갱신 방식(Weight Stationary, Input Stationary), partial sum(psum)의 축적 경로 등 데이터 재사용 구조를 정밀 분석한다.
- 메모리 계층(BRAM-register-local buffer) 간 데이터 이동량을 고려하여 FPGA에서의 연산 병목 요소를 규명한다.

(2) Single PE, 2×2, 3×3 구조의 Systolic Array 설계 및 구현

- 단일 Processing Element(PE) 기반 MAC 연산 구조를 Verilog로 구현한다.
- 이를 바탕으로 2×2 및 3×3 형태의 Systolic Array를 설계하여, 연산 병렬성(parallelism), 데이터 스트리밍 방향, 파이프라인 흐름의 변화를 비교할 수 있도록 구조적 일관성을 확보한다.
- 각 아키텍처는 FPGA 상에서 실제 동작하도록 제작되며, testbench를 통해 기능적 타당성을 검증한다.

(3) 구조별 연산 지연(latency) 및 처리량(throughput)의 정량 분석

- Single PE, 2×2, 3×3 구조 간 연산 완료까지 소요되는 clock cycle, 파이프라인 초기 지연(warm-up latency), steady-state 처리량을 비교·분석한다.
- weight 고정 여부, ifmap 스트리밍 방식, PE 개수 증가 등 구조적 설계 차이가 cycle-per-output, 데이터 이동량, 실제 연산 지연 시간에 미치는 영향을 정량적으로 측정한다.

- 이를 통해 CNN 연산의 하드웨어 가속 효과가 단순히 연산 자원의 증가가 아니라 데이터 이동 구조 최적화에 의해 주도됨을 실험적으로 확인한다.

2. Theoretical Approach

Efficient Processing of Deep Neural Networks – HW for DNN: Systolic Array [1]

딥러닝에서 Convolution Neural Network(CNN)는 ifmap(input feature map), filter weight, output feature map 사이에서 동일한 데이터가 여러 번 재사용되면서 연산이 반복되는 구조를 가진다. 하지만 일반적인 폰 노이만 구조에서는 데이터를 매번 DRAM에서 읽어와야 하고, 이 과정이 연산보다 훨씬 느리고 전력 소모도 크기 때문에 DNN 연산에서 병목현상을 일으킨다.

이를 완화하기 위해 Data Reuse를 극대화하는 여러 아키텍처가 제안되었고, 재사용하는 데이터의 종류에 따라 다음과 같이 구분할 수 있다.

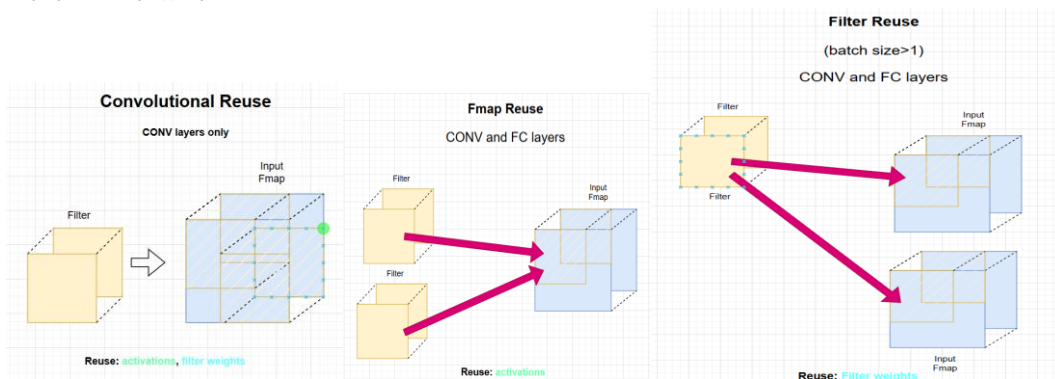


fig2.1,2,3) convolutional Reuse, Ifmap Reuse, Filter Reuse

(1) Convolutional Reuse (ifmap과 weight를 모두 재사용하는 방식)

(2) Ifmap Reuse (입력 특징맵(ifmap)의 한 pixel을 여러 channel/filter에서 반복 사용하는 방식)

(3) Filter Reuse (특정 filter weight를 batch 방향으로 여러 입력에 대해 재사용하는 방식)

이러한 재사용을 하드웨어로 구현하기 위해 사용하는 구조가 Systolic Array(SA)이다. SA는 PE(Processing Element)를 격자 형태로 배치하고, 각 PE에 작은 local memory(register)를 갖는다. Local memory를 통해 한 번만 메모리로부터 데이터를 가져오면, 이웃한 PE들 사이에서 데이터를 전달하여 여러 번 재사용할 수 있도록 한다.

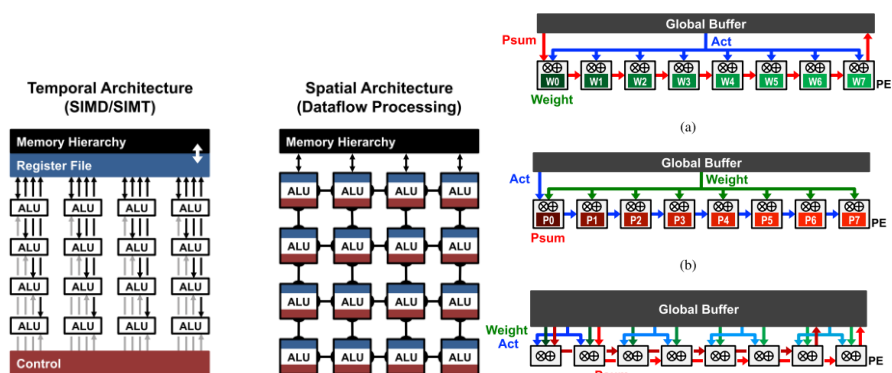


fig 2.4,5,6) Temporal architecture, Spatial Architecture, dataflow

딥러닝 연산 가속기의 하드웨어 구조는 크게 Temporal Architecture와 Spatial Architecture로 구분된다. Temporal 구조(SIMD/SIMT)는 단일 연산 유닛을 시간적으로 반복 사용하며 연산을 수행하는 방식으로, 레지스터 파일과 메모리 계층 사이의 지속적인 데이터 이동이 필요하여 메모리 접근이 병목으로 작용한다. 이 방식은 제어

오버헤드가 크고, 연산이 순차적으로 진행되기 때문에 대규모 병렬성이 필요한 CNN 연산에서는 처리 효율이 제한된다.

반면, Spatial Architecture는 연산 유닛(PE)을 2차원으로 배치하고 데이터가 이러한 배열 내부를 흐르듯 전달되며 계산이 이루어지는 dataflow 기반 구조이다. 이 방식은 연산에 필요한 데이터를 메모리로부터 반복적으로 로드하지 않고, 인접 PE 간 이동시키는 방식으로 공급하므로 데이터 재사용률이 높고 전력 소모가 크게 줄어든다. 특히 CNN의 convolution 연산처럼 동일한 weight와 입력 activation이 반복적으로 사용되는 연산에서 Spatial 구조는 매우 높은 처리량(throughput)을 제공한다.

여러 Spatial Data Flow 중에서도 본 프로젝트에서는 대표적인 Weight-Stationary 방식의 Systolic Array 구조를 선택하였다. Weight-Stationary 구조에서는 weight 값을 각 PE 내부에 고정적으로 저장하고, 입력 activation은 행(row) 방향으로 스트리밍되며, partial sum은 열(column) 방향으로 전파되도록 구성한다. 이러한 데이터 이동 방식은 convolution의 계산 패턴과 정확히 일치하며, 불필요한 메모리 접근을 제거함으로써 연산 효율을 극대화한다. 그림 (a)~(c)에서 제시된 세 가지 dataflow 가운데, (a)의 weight를 PE 내부에 고정하고 activation과 psum이 네트워크를 통해 진행하는 형태가 우리가 구현한 sys_33 설계의 핵심 동작 원리와 동일하다.

본 프로젝트는 Temporal Architecture의 메모리 왕복 병목을 회피하기 위해 Spatial Dataflow 기반의 Systolic Array를 채택하였으며, 3×3 convolution을 처리하는 3X3 systolic과 2×2 연산을 수행하는 2x2 systolic을 각각 설계하였다. sys_33은 weight를 PE 내부에 고정시키고 activation을 스트리밍하는 Weight-Stationary 구조를 구현하였고, sys_22는 Phase 기반 weight 로딩을 사용하여 작은 커널 이동에서도 높은 데이터 재사용률을 확보하였다. 두 구조 모두 PE 간 연속적인 psum 전달을 통해 파이프라인을 형성하며, convolution 연산의 구조적 특성을 하드웨어적으로 최적화할 수 있게 한다.

3. Computation Method Analysis

본 프로젝트에서 Convolution 연산은 세 가지 서로 다른 하드웨어 구조인 **Single_PE**, **sys_33**, **sys_22**를 통해 수행된다. 세 구조는 모두 동일한 연산 단위인 **PE** 모듈을 기반으로 구성되지만, PE의 개수와 데이터가 전달되는 방식이 서로 다르기 때문에 연산 지연(clock latency), weight 재사용 방식, ifmap 이동 방향 등이 크게 달라진다. 따라서 이 섹션에서는 각 구조가 어떤 원리로 연산을 수행하며, 왜 이러한 방식이 선택되었는지를 코드 흐름 중심으로 기술한다.

전체 시스템의 실행 순서는 **Controller** → **Memory** → **Computation(Single_PE, sys_33, sys_22)** → **Display**로 구성되어 있으며, Controller는 각 모듈에 active 신호를 제공하고, 각 모듈이 반환하는 done 신호를 받아 다음 상태로 전이한다. 즉, 실제 연산은 Controller의 FSM을 기준으로 정해진 순서대로 진행되며, 모든 Computation 모듈은 동일한 데이터 포맷(ifmap-weight-output)을 Memory로부터 공급받는다.

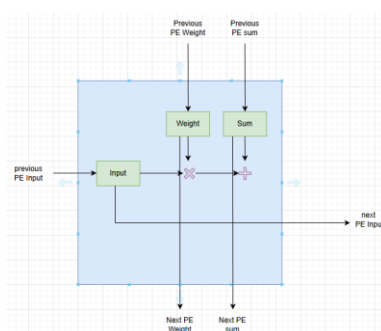


fig 3.1) PE module

본 프로젝트에서 구현한 Processing Element(PE)는 systolic array의 기본 계산 단위로서, 입력 데이터(input), 가중치(weight), 그리고 partial sum(psum)을 받아 multiply-accumulate(MAC) 연산을 수행하고, 다음 PE로 데이터를 전달하는 파이프라인 형태의 구조로 설계되었다. 하나의 PE는 입력 스트림을 받아 현재 위치의 weight와 곱한 뒤 이전 단계의 psum 값과 더하여 새로운 psum을 생성하며, 동시에 input과 weight를 지연 없이 다음 PE로 전달함으로써 systolic dataflow를 실현한다.

PE 내부는 크게 Input Register, Weight Register, Multiply Unit, Accumulate Unit으로 구성된다. 먼저 Input Register는 이전 PE 또는 Memory에서 공급되는 activation 값을 한 사이클 동안 저장하고, 해당 값을 multiply 연산과 다음 PE 전달 경로로 동시에 분기시킨다. Weight Register는 이전 PE 또는 weight shift register로부터 전달된 weight를 저장하여, multiply 단계에서 항상 고정된 weight 값을 사용할 수 있도록 한다. 이는 본 설계가 weight-stationary dataflow를 따르기 때문에 모든 weight가 각 PE 내부에 고정된 상태로 유지되는 것을 의미한다.

Multiply Unit에서는 Input Register에서 들어오는 activation 값과 Weight Register에 저장된 가중치가 곱해져 partial product가 생성된다. 이후 Accumulate Unit에서는 이전 PE로부터 전달된 psum 값이 입력되며, 이를 현재 partial product와 더하여 새로운 psum 값을 계산한다. 이 연산 결과는 현재 PE의 출력이자, 다음 PE의 accumulate 입력 신호로 전달되어 convolution의 MAC 구조가 연속적으로 이어지도록 한다.

또한 PE는 연산뿐 아니라 데이터 전달(systolic propagation) 역할을 수행한다. 입력 activation은 우측 방향으로 전달되고, weight는 아래 또는 오른쪽 방향으로 이동하며, psum은 통상 우측 또는 아래로 전달된다. 이를 통해 전체 systolic array는 외부 메모리 접근 없이 PE 간 데이터 이동만으로 연산이 진행되는 파이프라인 구조를 가지며, 각 PE는 “계산 + 전달”을 동시에 수행하는 자율 노드처럼 동작한다.

결과적으로 본 설계의 PE는 activation 입력, 고정 weight, psum 입력을 받아 MAC 연산을 수행하고, 세 종류의 데이터를 모두 다음 PE로 forwarding 하는 구조를 갖추고 있어, 3×3 및 2×2 systolic array 전체가 일정한 데이터 흐름 속에서 자연스럽게 convolution 연산을 수행할 수 있도록 지원한다.

1. Single_PE 구조 (Serial Calculation)

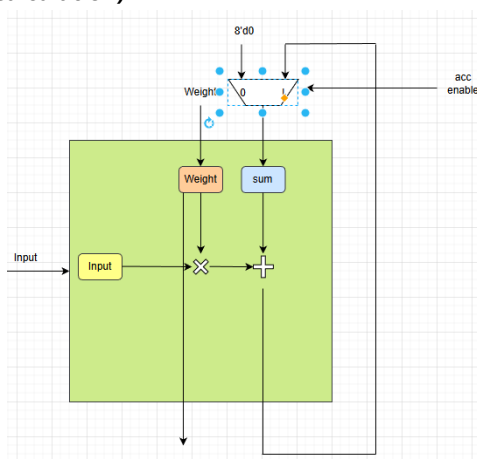


fig 3.2) single PE

본 연구에서는 systolic array 구조를 구현하기에 앞서, 단일 Processing Element(PE)를 이용한 **직렬(serial) 계산 방식**을 설계하였다. Single PE 구조는 multiply-accumulate 연산을 수행하는 MAC 기반 처리 유닛을 하나만 사용하여, 입력 데이터와 가중치를 순차적으로(시간축 기반) 처리하는 방식으로 동작한다. 이는 systolic array와 달리 공간적 병렬성을 사용하지 않고, 하나의 PE가 모든 연산을 시간에 따라 반복 수행한다는 점에서 차별적이다.

이러한 직렬 방식은 연산 단위가 적고 구조가 단순하다는 장점이 있으며, 전체 연산 흐름을 검증하고 PE 내부 동작을 독립적으로 테스트하는 데 효과적이다.

Single PE 내부는 Input Register, Weight Register, Sum Register, Multiply Unit, Accumulate Unit, 그리고 Accumulate Enable 제어 로직으로 구성된다. 입력 activation 값은 Input Register에 저장되며, weight 값은 Weight Register를 통해 주입된다. 두 데이터는 Multiply Unit에서 곱셈 연산을 수행하여 partial product를 생성한다. 이후 Accumulate Unit에서는 sum register에 저장된 이전 partial sum과 현재 partial product를 q더하여 새로운 누적 결과를 생성한다. 이 누적 값은 다시 sum register에 저장되어 다음 연산의 psum 입력으로 사용된다.

Single PE 직렬 구조에서 핵심은 **acc_active 신호와 MUX 초기값 선택 로직**이다. 직렬 방식에서는 첫 연산에서 기존 psum이 존재하지 않기 때문에, sum register의 초기값은 반드시 0이 되어야 한다. 이를 위해 acc_active 신호가 처음 사이클에서는 비활성화되며, MUX는 sum register 대신 상수값 0을 accumulate 입력으로 선택한다. 이후 acc_active 신호가 활성화되면, MUX는 sum register의 실제 psum 값을 전달하고, PE는 새로운 연산 결과를 누적해 나간다. 이 제어 구조는 convolution 연산에서 요구되는 MAC 반복을 단일 PE가 시간적으로 수행할 수 있도록 지원한다.

연산은 입력 데이터와 weight 값이 Memory 모듈로부터 순차적으로 공급되는 방식으로 진행된다. 예를 들어 2×2 또는 3×3 convolution window를 처리할 경우, Single PE는 각 위치의 activation × weight 연산을 순서대로 수행하고, sum register에 누적함으로써 최종 convolution 결과를 계산한다. 즉, systolic array가 공간적 병렬성을 통해 한 번에 여러 연산을 수행하는 것과 달리, Single PE는 동일한 기능을 **시간적 반복**을 통해 구현한다.

이 직렬 기반 PE 구조는 전체 systolic 시스템의 기본 연산 단위를 검증하기 위한 초기 단계로 활용되었으며, 이후 확장된 배열 구조(2×2, 3×3 systolic array)의 동작 원리를 이해하고 설계하는 데도 핵심적인 기반이 되었다.

2. systolic 3x3 구조 (3×3 Systolic Array : Weight Stationary 구조의 이상적인 형태)

Systolic 3x3는 3×3 커널과 동일한 크기의 3×3 PE 배열을 사용하며, Weight Stationary(WS) 방식이 가장 이상적으로 적용되는 구조이다. 연산 초기에 3 cycle 동안 weight가 행(row) 단위로 로딩되며, 첫 번째 cycle에는 w11, w12, w13이 상단 PE들에 저장되고, 두 번째 cycle에는 w21, w22, w23이 중단 PE들로, 세 번째 cycle에는 w31, w32, w33이 하단 PE들로 전달되어 전체 9개의 weight가 배열 내부에 고정된다.

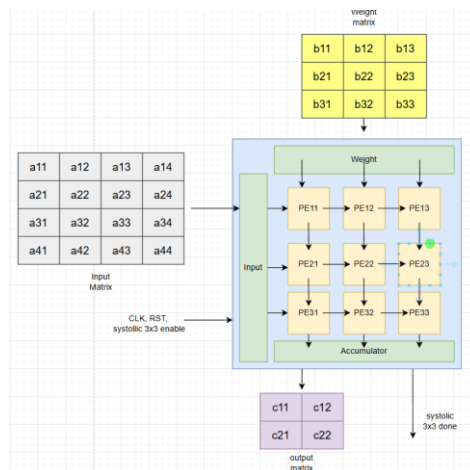


fig 3.3) 3x3 systolic array

weight가 고정된 이후에는 ifmap이 왼쪽과 상단에서 스트리밍 형태로 들어오며, 입력값은 우측·하단 방향으로 이동하고, partial sum 역시 인접한 PE로 전달되면서 누적된다. 특히 이 구조에서는 모든 PE가 매 사이클마다 유효한 연산을 수행하므로 pipeline이 깨지지 않으며, 세 구조 중 가장 작은 clock 수로 연산을 완료할 수 있다. done 신호는 마지막 PE(우하단 PE)에서 누산이 종료된 시점에서 발생하도록 구현되어 있다.

3. systolic 2x2 구조 (2x2 Systolic Array : WS 변형 구조, Phase 기반 weight 로딩 방식)

systolic 2x2는 총 4개의 PE로 구성된 2x2 Systolic Array이다. PE의 수가 3x3 커널 전체를 담기에는 부족하기 때문에 sys_22는 Phase 기반 weight 재로딩 방식을 사용한다.

Phase 1에서는 커널 좌상단의 {w11, w12, w21, w22}가 PE들에 로딩되며, Phase 2에서는 커널의 오른쪽으로 이동한 부분인 {w12, w13, w22, w23}, Phase 3에서는 아래로 이동한 부분인 {w21, w22, w31, w32}가 다시 로딩된다.

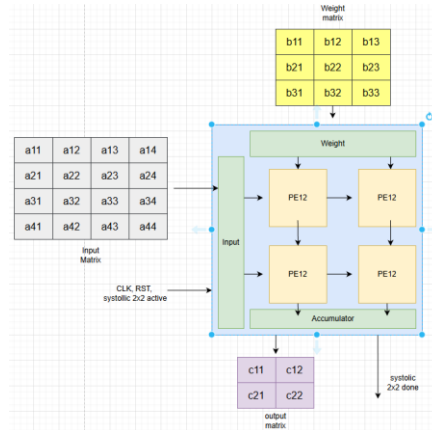


fig3.4) 2x2 systolic array

각 Phase마다 weight를 새로 공급받고, ifmap 역시 해당 커널 위치에 맞춰 다시 스트리밍되기 때문에 3x3 SA(sys_33)에 비해 weight/ifmap 이동이 많고 pipeline이 매번 끊기게 된다. 따라서 연산 시간이 증가하지만, Single_PE보다는 훨씬 효율적이며 제한된 PE 수로 3x3 커널을 처리할 수 있다는 장점이 있다.

Verilog 코드(sys_22)는 이 과정을 매우 세밀한 FSM으로 구성하며, 각 Phase별 weight 입력 timing, ifmap shifting timing, psum 누산 timing이 state 단위로 정확히 구분되어 있다.

4. Module Design Analysis

4.1) Top module

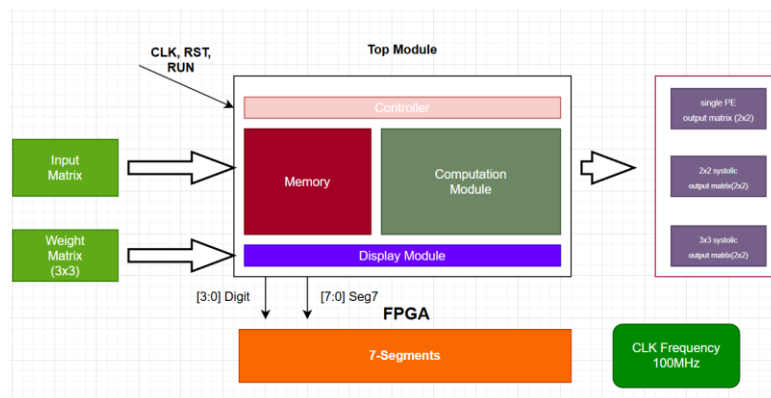


fig 4.1) top module diagram

본 프로젝트의 Top Module은 FPGA 내부에서 이루어지는 전체 연산 구조의 최상위 계층으로, Controller, Memory, Computation Module, Display Module을 하나의 시스템으로 통합하는 중심 역할을 수행한다. 외부로부터는 입력 행렬과 가중치 행렬을 받아들이고, 내부적으로는 클럭(CLK), 초기화(RST), 실행 신호(RUN)에 따라 각 모듈이 순차적으로 동작하도록 제어한다. Top Module은 이러한 모듈 간 인터페이스를 모두 연결하여 전체 시스템의 데이터 흐름과 연산 절차가 올바르게 진행되도록 설계되었다.

우선 Top Module의 가장 상단에는 Controller가 위치하며, 이 블록은 전체 연산의 흐름을 스케줄링하는 역할을 담당한다. RUN 신호가 입력되면 Controller는 Memory와 Computation Module이 어떤 연산 모드로 작동할지를 결정하기 위한 active 신호를 생성하고, 각 모듈이 필요한 시점에 활성화되도록 타이밍을 조절한다. Controller 내부의 FSM은 single PE 연산, 2x2 systolic 연산, 3x3 systolic 연산 중 선택된 모드에 따라 필요한 cycle 수를 카운트하고, 모든 연산이 종료되었을 때 DONE 신호를 발생시켜 다음 단계로 넘어갈 수 있게 한다.

Memory 모듈은 Top Module의 핵심 데이터 저장 및 재배치 블록으로, 입력된 4x4 Activation 행렬(a11~a44)과 3x3 Weight 행렬(b11~b33)을 내부 버퍼에 저장한 뒤, 선택된 연산 방식에 맞추어 데이터를 다시 구성하여 Computation Module로 공급한다. 예를 들어 3x3 systolic 연산이 수행되는 경우 Memory는 activation을 행 단위(row-buffer)로 직렬화하여 PE 배열의 좌측 입력에 순차적으로 스트리밍하고, weight는 b11부터 b33까지 PE의 위치에 대응되는 순서로 출력하여 w_shift_reg로 로딩된다. 2x2 systolic 모드에서는 phase 단위의 입력 및 가중치 조합을 출력하며, single PE 모드에서는 단일 window 데이터만을 바로 전달한다. Memory는 단순 저장 기능을 넘어서 각 연산 모드에 최적화된 데이터 스케줄링을 수행하며, Controller의 enable/timing 신호에 따라 동적인 출력 패턴을 생성한다.

Computation Module은 Memory로부터 전달된 activation 및 weight 데이터를 바탕으로 실제 연산을 수행하는 연산 엔진이다. 이 모듈은 세 가지 연산 방식을 모두 포함하는 구조로 설계되어 있으며, single PE 방식에서는 단일 2x2 window를 이용한 기본 연산을 수행하고, 2x2 systolic 모드에서는 PE 배열의 상호 연결을 이용해 파이프라인 형태의 연산을 진행하며, 3x3 systolic 모드에서는 3x3 PE 배열 전체를 활용하여 convolution 연산을 하드웨어적으로 구현한다. 각 모드에서 결과(c11, c12, c21, c22)가 생성되면 Computation Module은 이를 Display Module로 전달하고, 동시에 Controller에 DONE 신호를 보내 연산 종료를 알린다.

Display Module은 Computation Module에서 생성된 결과 값을 실제 보드의 7-Segment Display로 출력하는 기능을 담당한다. 이 모듈은 연산된 값들을 7-Segment가 해석할 수 있는 segment 패턴으로 변환하고, 자릿수 선택 신호와 segment 라인을 생성하여 FPGA 외부 핀으로 출력한다. Artix-7 보드에 탑재된 7-Segment는 공통 애노드 방식으로 동작하므로 Display Module은 100 MHz 시스템 클럭을 분주하여 time-division multiplexing 방식으로 각 자릿수를 빠르게 스캔하며, 결과적으로 네 자리의 출력이 동시에 켜져 보이는 효과를 만든다. 이를 통해 systolic array의 연산 결과는 사용자에게 직관적으로 확인 가능한 형태로 표시된다.

전체적으로 Top Module은 각 모듈이 독립적으로 기능하면서도 하나의 연산 경로를 형성할 수 있도록 모든 요소를 조정하는 총괄적인 통합 블록으로 설계되었다. 입력 데이터의 로딩, 모드 설정, 데이터 스케줄링, 연산 실행, 연산 결과 출력의 모든 과정은 Top Module 내부에서 완전한 파이프라인 구조로 연결되어 있으며, 이를 통해 FPGA 상에서 systolic array 기반의 연산 시스템이 안정적으로 동작하도록 구현하였다.

4.2) controller module

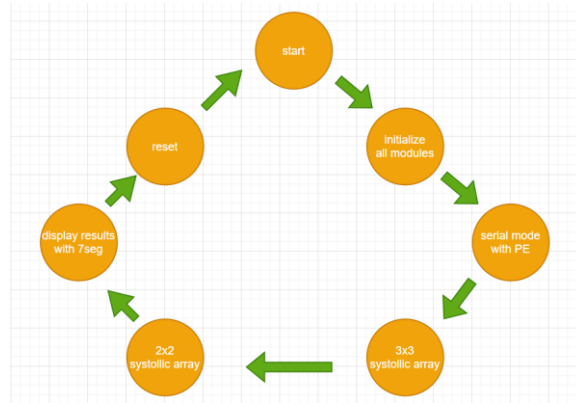


fig 4.2) controller module state diagram

본 시스템의 Controller 모듈은 Top Module 내부에서 전체 연산 순서를 결정하는 핵심 제어 블록으로, 입력 초기화부터 systolic array 연산, 그리고 결과 출력과 리셋에 이르기까지 모든 동작의 흐름을 FSM(Finite State Machine) 구조로 관리한다. Controller는 클럭(100 MHz)과 외부의 RUN, RST 신호를 기반으로 상태 전이를 수행하며, 각 상태에서 필요한 active 신호를 Memory, Computation Module, Display Module로 전달한다. 그림에 나타난 흐름(start → initialize → serial mode → 3×3 systolic → 2×2 systolic → display → reset)은 Controller FSM의 전체 동작 과정을 단계적으로 표현한 것이다.

시스템이 실행되면 Controller는 가장 먼저 START 상태에 진입하며, 외부 RUN 신호가 활성화되는 시점을 기다린다. RUN 신호가 들어오면 FSM은 즉시 다음 단계인 모듈 초기화 상태로 이동하고, 이 단계에서 Memory, Computation Module, Display Module의 내부 레지스터와 제어 플래그를 초기화한다. 모든 모듈이 초기값을 갖추면 연산 준비가 완료되었다는 의미이므로 FSM은 다음 연산 모드로 이동한다.

초기화가 완료되면 Controller는 serial mode with PE 상태로 전환되며, 이 단계에서 단일 PE를 이용한 기본적인 직렬 연산 테스트 혹은 단순한 2×2 연산 결과를 발생시키기 위한 데이터 스트리밍을 Memory 모듈에 지시한다. 이 단계의 목적은 systolic 구조에 진입하기 전, 입력과 가중치가 정상적으로 전달되는지, 그리고 기본적인 Multiply-Accumulate 동작이 올바르게 이루어지는지를 확인하는 것이다. 연산이 정상적으로 완료되면 FSM은 다음 단계인 systolic 연산 단계로 이동할 준비를 한다.

다음 단계는 3×3 systolic array 연산 상태이다. 이 단계에서 Controller는 Memory 모듈에 3×3 systolic 연산에 필요한 row-buffer 기반 activation 스트리밍과 weight 공급을 지시한다. 동시에 Computation Module 내의 3×3 PE 배열을 활성화하는 sys33_active 신호를 발생시킨다. 해당 단계에서는 convolution 연산이 파이프라인 구조로 진행되며, 모든 출력(c11, c12, c21, c22)이 계산되면 Computation 모듈은 sys33_done 신호를 Controller로 보내고, 이를 수신한 FSM은 다음 단계로 전환한다.

3×3 연산이 종료되면 FSM은 2×2 systolic array 연산 상태로 진입한다. 이 상태에서는 Phase 기반 weight 로딩을 포함한 2×2 systolic 연산 로직이 활성화되며, 컨트롤러는 sys22_active 신호를 통해 Computation Module의 2×2 데이터 경로를 선택한다. Memory 모듈은 Controller의 모드 신호를 기반으로 activation과 weight를 다시 구성하여 내보내며, 모든 phase가 완료되면 sys22_done 신호가 Controller에 전달된다.

모든 연산이 완료되면 FSM은 display results with 7-seg 상태에 진입한다. 이 단계에서 Controller는 Display Module에 display_active 신호를 제공하여 최종 연산 결과를 BCD 혹은 segment code로 변환한 뒤 FPGA의 외부 7-Segment에 출력하도록 지시한다. 이 단계는 사용자에게 계산된 결과를 확인시키는 역할을 하며, 결과 출력이 안정화되면 최종적으로 다음 단계로 전환된다.

마지막 단계는 reset 상태이다. Controller는 전체 모듈의 내부 신호를 초기 상태로 되돌리기 위한 reset pulse를 발생시키고, Memory 버퍼, register, computation 내부 accumulator 등을 다시 초기화한다. 리셋이 완료되면 FSM은 다시 START 상태로 복귀하여 새로운 RUN 신호를 대기한다. 이 과정을 반복하면서 시스템은 연속적인 연산 사이클을 수행할 수 있다.

Controller 모듈은 이처럼 전체 시스템의 흐름을 시퀀스 형태로 완전히 제어하며, 각 모듈이 적절한 시점에만 동작하도록 조율함으로써 systolic array 기반 구조가 정확하고 효율적으로 동작하도록 한다.

4.3) Memory module

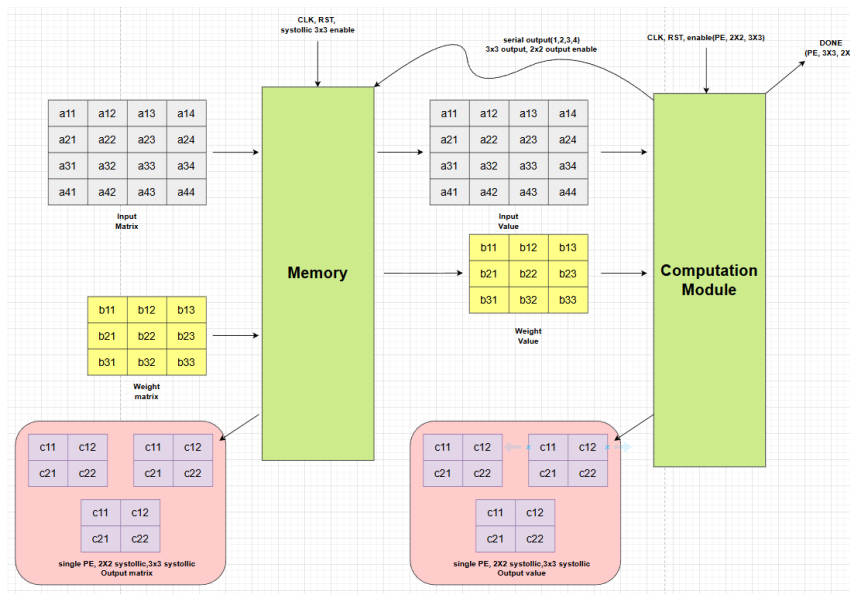


fig4.3) memory module dataflow

본 프로젝트에서 구현한 Memory 모듈은 입력 행렬(Input Matrix)과 가중치 행렬(Weight Matrix)을 버퍼링(buffering)하고, 선택된 연산 모드(single PE, 2x2, 3x3 systolic)에 맞추어 데이터를 순차적으로 제공하는 역할을 수행한다. 즉, Memory는 외부 입력을 단순 저장할 뿐만아니라, 실제 연산 흐름에 맞추어 데이터를 재배치, 스트리밍, 분배(distribution)하는 combinational addressing logic으로 설계하였다.

우선 Input Matrix(a11~a44)와 Weight Matrix(b11~b33)는 Memory에 저장된 후, systolic array 또는 단일 PE가 요구하는 순서에 따라 row-buffer / column-buffer 기반 스트리밍 형태로 제공된다. 3x3 systolic 연산이 활성화된 경우, Memory는 activation 값을 a11 → a12 → a13 순으로 1행부터 공급하며, 이후 2행(a21~a23), 3행(a31~a33)까지 순차적으로 출력한다. 이 과정에서 activation 데이터는 systolic array의 좌측 입력 포트 형태에 정확히 맞게 직렬(serial) 형태로 변환되어 전달된다.

가중치 값 역시 연산 모드에 따라 서로 다른 방식으로 출력된다. 3x3 모드에서는 b11~b33을 PE 위치에 대응되는 순서로 연속적으로 출력하며, 2x2 모드에서는 Phase 1~3에 해당하는 weight subset을 반복해서 출력한다. Memory는 이러한 가중치 출력 순서를 모두 내부 로직으로 관리하며, 각각의 weight는 computation module 내부의 w_shift_reg로 로드된 후 고정된다(Weight-Stationary).

또한 Memory 모듈은 단순한 값 출력뿐 아니라 systolic 연산 제어를 위한 여러 신호도 함께 생성한다. 그리고 제어 신호를 통해 computation module이 어떤 데이터 세트를 사용할지 결정하도록 한다.

결과적으로 Memory 모듈은 단순 RAM이 아니라, Input, weight data를 systolic array가 필요로 하는 순서와 패턴으로 재구성하여 공급하는 system 이자, 단일 PE, 2×2 , 3×3 연산 환경 모두에 공통적으로 대응할 수 있는 중앙 데이터 관리 역할을 수행한다.

4.4) display module

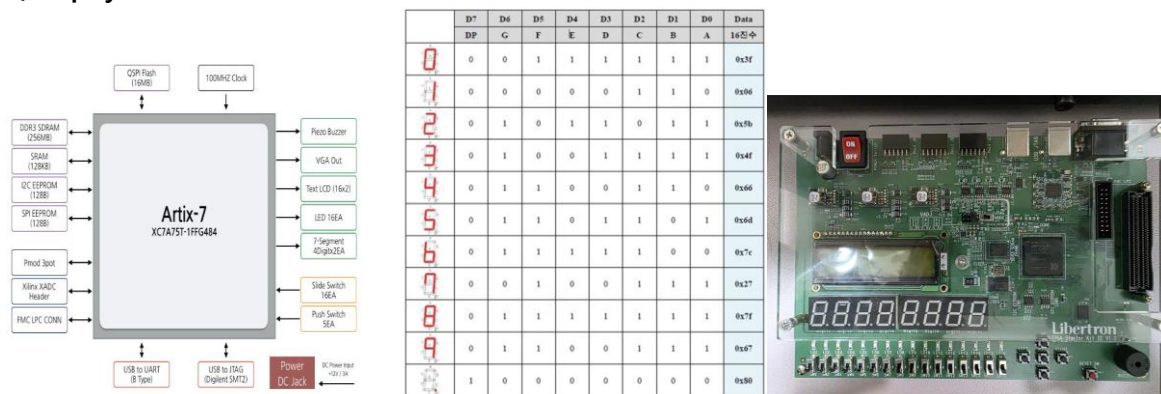


fig4.4) brief schematic circuit of Artix-7 XC7A75T / 7-seg hex table / FPGA kit

본 fpga kit의 datasheet를 참고한 결과, 본 실습에서는 별도의 외부 회로를 설계하지 않고, Artix-7 FPGA 보드에 이미 탑재된 7-Segment Display를 사용하였다. 우리는 Verilog로 연산 및 7-Segment 제어 로직만을 작성하였고, 실제 숫자 표시까지의 과정은 다음과 같은 순서에 맞게 1초 간격으로 출력하도록 설계하였다.

1. Verilog 설계 및 시뮬레이션

먼저 systolic array, controller, 7-Segment driver 등 모든 모듈을 Verilog로 작성하고, 시뮬레이션을 통해 연산 결과와 seg/an 신호가 의도대로 생성되는지 확인한다. 이 단계까지는 순수하게 “코드 수준”에서의 검증이다.

2. 합성(Synthesis) 및 구현(Implementation)

작성한 Verilog 코드는 Vivado 툴에서 Artix-7(XC7A75T-1FFG484)을 대상으로 합성(synthesis)된다. 이 과정에서 HDL로 표현된 설계가 LUT, 플립플롭, 배선 등 FPGA 내부 자원으로 변환된다. 이어서 구현(implementation) 단계에서 실제 핀 배치와 배선이 결정되고, 7-Segment에 연결된 보드 핀 위치는 XDC 제약 파일을 통해 FPGA의 출력 포트(seg[6:0], an[3:0])와 매핑된다.

3. 비트스트림(bitstream) 생성 및 다운로드

구현이 완료되면 Vivado는 FPGA 설정 정보를 담은 bitstream 파일(.bit)을 생성한다. 우리는 USB-JTAG 인터페이스를 통해 이 bitstream을 Artix-7 칩 내부의 구성 메모리로 다운로드한다. 이 시점부터 FPGA 내부의 하드웨어 구조가 우리가 설계한 회로 형태로 재구성된다.

4. FPGA 하드웨어로의 변환 및 동작 시작

bitstream이 로드되면 Artix-7 내부에는 systolic array, FSM, 7-Segment driver가 실제 하드웨어 회로로 존재하게 된다. 보드의 100 MHz 시스템 클럭과 전원(DC Jack)을 기반으로, 이 회로들은 클럭 에지마다 동작하며 연산 결과를 생성하고, 그 결과를 7-Segment용 출력 신호(seg/an)로 변환한다.

5. 보드 레벨에서의 7-Segment 점등

FPGA 핀에서 출력된 seg/an 신호는 보드에 미리 연결되어 있는 7-Segment 모듈로 전달된다. 각 세그먼트 라인과 자릿수 선택 라인은 보드의 저항·트랜지스터 등을 거쳐 실제 LED에 전류를 공급한다.

해당 아날로그 회로를 직접 설계하지는 않았지만, 보드 설계가 이미 완료되어 있기 때문에 FPGA가 1/0 신호만 올바르게 출력하면 해당 세그먼트 LED가 켜지게 된다.

결과적으로, 우리가 Verilog로 정의한 숫자 인코딩과 시간 다중화(multiplexing) 로직이 FPGA 하드웨어에서 실행되고, 그 신호가 보드 상의 7-Segment에 전달되면서 최종적으로 사람이 볼 수 있는 숫자 패턴이 표시된다.

정리하자면, Vivado의 합성·구현·bitstream 다운로드 과정을 거치면서 이 코드가 Artix-7 내부의 실제 하드웨어 회로로 변환된다. 이후 보드에 설계되어 있는 7-Segment 구동 회로가 이 신호를 받아 LED를 켜 주기 때문에, 별도의 추가 하드웨어 없이도 코드만으로 숫자 출력이 가능하다.

5. Verilog Implementation & Simulation Result

5.1) PE

a. code

```

3 module PE (
4     input clk, rst,
5     input filter_active, matrix_active, sum_active,
6
7     input [7:0] sum_in,
8     input [7:0] matrix_in, filter_in,
9
10    output [7:0] matrix_out, filter_out,
11    output [7:0] sum_out
12);
13
14    // Internal computation wires
15    wire [7:0] mul_out;
16    wire [7:0] sum_next;
17
18    // Register storage (Buffers)
19    wire [7:0] matrix_buffer;
20    wire [7:0] filter_buffer;
21    wire [7:0] sum_buffer;
22
23    // Matrix (Activation) Register
24    Register matrix_reg (
25        .clk(clk),
26        .rst(rst),
27        .active(matrix_active),
28        .in(matrix_in),
29        .out(matrix_buffer)
30    );
31
32    // Filter (Weight) Register
33    Register filter_reg (
34        .clk(clk),
35        .rst(rst),
36        .active(filter_active),
37        .in(filter_in),
38        .out(filter_buffer)
39    );
40
41    // Partial Sum Register
42    Register sum_reg (
43        .clk(clk),
44        .rst(rst),
45        .active(sum_active),
46        .in(sum_in),
47        .out(sum_buffer)
48    );
49
50    // Multiplier: matrix x filter
51    Multiplier_8bit multiplier (
52        .a (matrix_buffer),
53        .b (filter_buffer),
54        .cout(mul_out)
55    );
56
57    // Adder: mul_out + sum_buffer
58    Adder_8bit adder (
59        .a (mul_out),
60        .b (sum_buffer),
61        .cin(1'b0),
62        .sum(sum_next),
63        .cout()
64    );
65
66    // Outputs to next PE
67    assign matrix_out = matrix_buffer;
68    assign filter_out = filter_buffer;
69    assign sum_out = sum_next;
70
71 endmodule
72

```

fig5.5) PE Verilog Code 1, 2, 3, 4

위에서 제시한 PE 다이어그램을 위와 같은 코드로 구현하였다. 3개의 레지스터(matrix_reg, filter_reg, sum_reg)를 통해 PE의 가장 핵심적인 특징인 activation, weight, partial sum을 PE 내부에서 저장하고 인접한 PE로 전달해주는 기능을 구현하였다. 각각의 레지스터들은 active신호를 통해 제어되며, 각 레지스터의 buffer가 multiplier모듈과 adder모듈을 통해 곱셈 및 누산 과정을 거치게 된다. Outputs으로 해당 PE 내부에 저장되어 있던 activation, weight 그리고 연산 결과인 partial sum이 출력된다. 연산 과정에서 8bit multiplier와 8bit adder를 사용하기 때문에 8비트 이상의 overflow는 출력되지 않는다.

b. simulation result

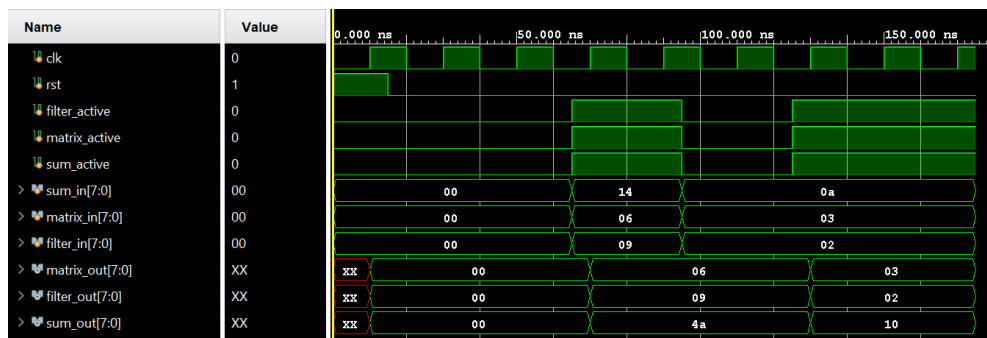


fig5.6) PE Testbench Waveform

테스트벤치 시뮬레이션 결과를 보면, 각 레지스터의 active 신호가 활성화될 때, 연산 결과와 matrix, filter값이 정상적으로 저장되고 out을 통해 출력되는 것을 확인할 수 있다. 또한 active신호가 비활성화 되어있을때는 새로운

matrix나 filter값이 들어오더라도 출력값들이 변하지 않다가, active신호가 활성화 될 때, 성공적으로 연산 결과, matrix, filter값들을 출력하는 것을 볼 수 있다.

5.2) Single PE

a. code

```

1 module Single_PE;
2     input clk, rst, pe_active;
3
4     input [7:0] a11, a12, a13, a14,
5           a21, a22, a23, a24,
6           a31, a32, a33, a34,
7           a41, a42, a43, a44;
8
9     input [7:0] b11, b12, b13,
10           b21, b22, b23,
11           b31, b32, b33;
12
13     output reg s_active;
14     output reg [7:0] s_c11, s_c12, s_c21, s_c22;
15     output reg pe_done;
16
17
18
19
20
21     localparam S0=0, S1=1, S2=2, S3=3, S4=4, S5=5, S6=6, S7=7, S8=8, S9=9,
22           S10=10, S11=11, S12=12, S13=13, S14=14, S15=15, S16=16, S17=17, S18=18, S19=19,
23           S20=20, S21=21, S22=22, S23=23, S24=24, S25=25, S26=26, S27=27, S28=28, S29=29,
24           S30=30, S31=31, S32=32, S33=33, S34=34, S35=35, S36=36, S37=37, S38=38;
25
26     wire [7:0] sum;
27     wire [7:0] acc_sum;
28     reg acc_active;
29
30     reg [7:0] matrix_in;
31     reg [7:0] filter_in;
32
33     reg [5:0] state, next_state;
34
35     PE pe (
36         .clk (clk),
37         .rst (rst),
38         .filter_active (1'b1),
39         .matrix_active (1'b1),
40         .sum_active (1'b1),
41         .sum_in (acc_sum),
42         .matrix_in (matrix_in),
43         .filter_in (filter_in),
44         .matrix_out (),
45         .filter_out (),
46         .sum_out (sum)
47     );
48
49     assign acc_sum = (acc_active) ? sum : 8'd0;

```

fig5.7) Single PE Verilog Code 1, 2

PE를 하나만 사용하여 직렬 계산을 구현하기 위해 위와 같이 변수들을 설정하였다. pe_active 신호를 computation module로부터 받아와서 직렬 계산을 시작하게 된다. s_active 신호는 모든 계산이 종료된 후 s_c11~s_c22에 저장되어 있는 직렬 계산 결과 행렬(2x2)을 memory에 저장하도록 하는 신호이다. acc_active 신호는 누적합 지속 여부를 결정하는 중요한 신호이다. 결과 행렬의 한 요소가 누적합을 통해 완성될 때까지, 한번의 PE연산 결과인 sum이 다음 PE연산의 sum_in으로 들어간다. 만약 결과 행렬의 한 요소(ex. s_c11)가 완성되면 acc_active신호를 비활성화하여 다음 PE연산의 acc_sum값을 0으로 초기화한다. 직렬 계산은 systolic array와 다르게 그 어떤 weight값도 PE내부에서 고정하지 않기 때문에, filter_active, matrix_active, sum_active 신호를 활성화하여 PE내부로 새로운 값이 입력되면 해당 값으로 연산과 누적합을 진행한다.

```

51     always @(posedge clk) begin
52         if (rst) begin
53             state <= S0;
54         end else begin
55             state <= next_state;
56         end
57     end
58
59     always @(*) begin
60         next_state = state;
61
62         acc_active = 1'b0;
63         s_active = 1'b0;
64         pe_done = 1'b0;
65         matrix_in = 8'd0;
66         filter_in = 8'd0;
67
68         case (state)
69             S0: begin
70                 if (pe_active)
71                     next_state = S1;
72             end
73             S1: begin
74                 next_state = S2;
75             end
76             S2: begin
77                 // C11
78                 next_state = S3;
79                 acc_active = 1'b0;
80                 matrix_in = a11;
81                 filter_in = b33;
82             end
83             S3: begin
84                 next_state = S4;
85                 acc_active = 1'b1;
86                 matrix_in = a12;
87                 filter_in = b32;
88             end
89             S4: begin
90                 next_state = S5;
91                 acc_active = 1'b1;
92                 matrix_in = a13;
93                 filter_in = b31;
94             end
95             S5: begin
96                 next_state = S6;
97                 acc_active = 1'b1;
98                 matrix_in = a21;
99                 filter_in = b23;
100
101             S11: begin
102                 next_state = S12;
103                 acc_active = 1'b0;
104                 s_c11 <= sum;
105                 matrix_in = a12;
106                 filter_in = b33;
107             end
108             S12: begin
109                 next_state = S13;
110                 acc_active = 1'b1;
111                 matrix_in = a13;
112                 filter_in = b32;
113             end
114             S13: begin
115                 next_state = S14;
116                 acc_active = 1'b1;
117                 matrix_in = a14;
118                 filter_in = b31;
119             end
120             S14: begin
121                 next_state = S15;
122                 acc_active = 1'b1;
123                 matrix_in = a22;
124                 filter_in = b23;
125             end
126             S15: begin
127                 next_state = S16;
128                 acc_active = 1'b1;
129                 matrix_in = a23;
130                 filter_in = b22;
131             end
132             S16: begin
133                 next_state = S17;
134                 acc_active = 1'b1;
135                 matrix_in = a24;
136                 filter_in = b21;
137             end
138             S17: begin
139                 next_state = S18;
140                 acc_active = 1'b1;
141                 matrix_in = a31;
142                 filter_in = b13;
143             end
144             S18: begin
145                 next_state = S19;
146                 acc_active = 1'b1;
147                 matrix_in = a32;
148                 filter_in = b12;
149             end
150             S19: begin
151                 next_state = S20;
152                 acc_active = 1'b1;
153                 matrix_in = a33;
154                 filter_in = b11;
155             end
156             S20: begin
157                 next_state = S21;
158                 acc_active = 1'b1;
159                 matrix_in = a34;
160                 filter_in = b11;
161             end
162             S21: begin
163                 next_state = S22;
164                 acc_active = 1'b1;
165                 matrix_in = a41;
166                 filter_in = b11;
167             end
168             S22: begin
169                 next_state = S23;
170                 acc_active = 1'b1;
171                 matrix_in = a42;
172                 filter_in = b11;
173             end
174             S23: begin
175                 next_state = S24;
176                 acc_active = 1'b1;
177                 matrix_in = a43;
178                 filter_in = b11;
179             end
180             S24: begin
181                 next_state = S25;
182                 acc_active = 1'b1;
183                 matrix_in = a44;
184                 filter_in = b11;
185             end
186             S25: begin
187                 next_state = S26;
188                 acc_active = 1'b1;
189                 matrix_in = a44;
190                 filter_in = b11;
191             end
192             S26: begin
193                 next_state = S27;
194                 acc_active = 1'b1;
195                 matrix_in = a44;
196                 filter_in = b11;
197             end
198             S27: begin
199                 next_state = S28;
200                 acc_active = 1'b1;
201                 matrix_in = a44;
202                 filter_in = b11;
203             end
204             S28: begin
205                 next_state = S29;
206                 acc_active = 1'b1;
207                 matrix_in = a44;
208                 filter_in = b11;
209             end
210             S29: begin
211                 next_state = S30;
212                 acc_active = 1'b1;
213                 matrix_in = a44;
214                 filter_in = b11;
215             end
216             S30: begin
217                 next_state = S31;
218                 acc_active = 1'b1;
219                 matrix_in = a44;
220                 filter_in = b11;
221             end
222             S31: begin
223                 next_state = S32;
224                 acc_active = 1'b1;
225                 matrix_in = a44;
226                 filter_in = b11;
227             end
228             S32: begin
229                 next_state = S33;
230                 acc_active = 1'b1;
231                 matrix_in = a44;
232                 filter_in = b11;
233             end
234             S33: begin
235                 next_state = S34;
236                 acc_active = 1'b1;
237                 matrix_in = a44;
238                 filter_in = b11;
239             end
240             S34: begin
241                 next_state = S35;
242                 acc_active = 1'b1;
243                 matrix_in = a44;
244                 filter_in = b11;
245             end
246             S35: begin
247                 next_state = S36;
248                 acc_active = 1'b1;
249                 matrix_in = a44;
250                 filter_in = b11;
251             end
252             S36: begin
253                 next_state = S37;
254                 acc_active = 1'b1;
255                 matrix_in = a44;
256                 filter_in = b11;
257             end
258             S37: begin
259                 next_state = S38;
260                 acc_active = 1'b1;
261                 matrix_in = a44;
262                 filter_in = b11;
263             end
264             S38: begin
265                 next_state = S0;
266                 s_active = 1'b1;
267                 s_c22 <= sum;
268                 pe_done = 1'b1;
269             end
270         endcase
271     end
272 endmodule

```

fig5.8) Single PE Verilog Code 3, 4, 5, 6

직렬 계산은 위와 같이 현재 상태만에 따라서 출력을 결정하는 무어(Moore)머신 방식 FSM으로 구현하였다. 결과 행렬인 s_c11, s_c12, s_c21, s_c22를 순차적으로 완성시키도록 각 state마다 activation과 weight를 설정하였다. 한 결과 행렬 요소를 만들기 위한 모든 계산이 완료되어 해당 결과가 sum에 있다면, sum값은 s_c11과 같이 해당 결과 행렬 요소에 저장되고, acc_active 신호를 비활성화하여 누적합을 초기화한다. 한 결과 요소당 이 과정이 9번 반복되어야 하므로, 총 36번의 반복 후, s_active 신호와 pe_done 신호를 활성화 하여, memory와 computation module에 직렬 계산이 완료되었음을 알린다.

b. simulation result

```

68 : ○ initial begin
69 :     // 4x4 matrix A
70 : ○   a11=8'd1; a12=8'd2; a13=8'd3; a14=8
71 :     a21=8'd5; a22=8'd6; a23=8'd7; a24=8
72 :     a31=8'd9; a32=8'd10; a33=8'd11; a34=8
73 :     a41=8'd13; a42=8'd14; a43=8'd15; a44=8
74 :
75 :     // 3x3 filter B
76 : ○   b11=8'd1; b12=8'd2; b13=8'd3;
77 : ○   b21=8'd4; b22=8'd5; b23=8'd6;
78 : ○   b31=8'd7; b32=8'd8; b33=8'd9;
79 : ○ end
80 :
81 : endmodule

```

fig5.9) Single PE Verilog Code 7

위와 같이 input 행렬과 filter 행렬을 투입하여 Single PE 과정이 정상적으로 작동하는지 확인하였다.

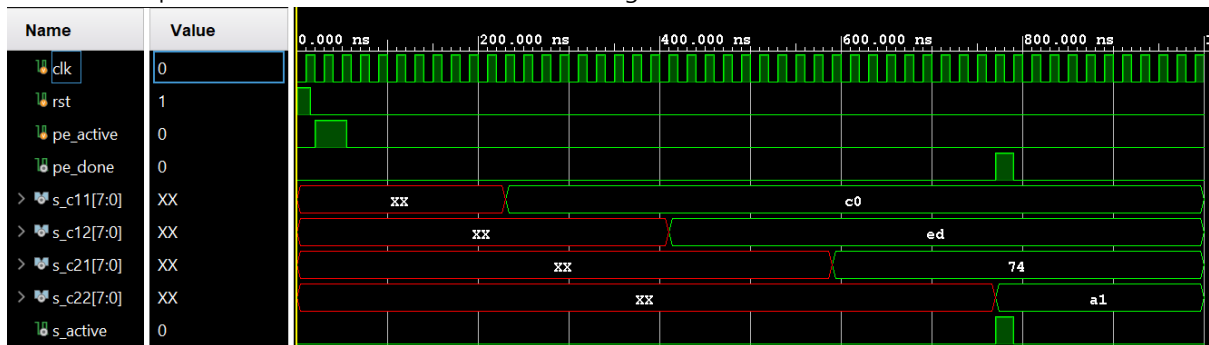


fig5.10) Single PE Testbench Waveform

pe_active 신호가 입력되면 직렬 계산이 시작되고, 각 결과 요소(s_c11~s_c22)가 순차적으로 정확하게 계산되는 것을 확인할 수 있었다. 모든 계산이 완료된 후에도 pe_done과 s_active 신호가 활성화되는 것도 확인할 수 있었다. 이를 통해 computation module과 controller는 다음 systolic array 계산과정으로 넘어가는 동작을 할 것이고, memory에는 직렬 계산 결과가 저장될 것임을 예측할 수 있다.

5.3) Systolic Array 3x3

a. code

```

55 in_shift_register append_pipe_row1(.clk(clk), .rst(rst), .active(buffer_active), .shift(in_shift[0]), .in(matrix_buffer1), .out(pipe_row1[0]));
56 in_shift_register append_pipe_row2(.clk(clk), .rst(rst), .active(buffer_active), .shift(in_shift[1]), .in(matrix_buffer2), .out(pipe_row2[0]));
57 in_shift_register append_pipe_row3(.clk(clk), .rst(rst), .active(buffer_active), .shift(in_shift[2]), .in(matrix_buffer3), .out(pipe_row3[0]));
58
59 w_shift_register append_pipe_col1(.clk(clk), .rst(rst), .active(buffer_active), .shift(w_shift), .in(filter_buffer1), .out(pipe_col1[0]));
60 w_shift_register append_pipe_col2(.clk(clk), .rst(rst), .active(buffer_active), .shift(w_shift), .in(filter_buffer2), .out(pipe_col2[0]));
61 w_shift_register append_pipe_col3(.clk(clk), .rst(rst), .active(buffer_active), .shift(w_shift), .in(filter_buffer3), .out(pipe_col3[0]));

```

fig5.11) Systolic Array 3x3 Verilog Code 1

systolic 3x3 계산에서는 효율적인 연산을 위하여 weight를 memory로부터 불러온 후, 9개의 PE 내부에 저장 및 고정한다. 연산을 진행할 때, 각 PE에서 고정된 weight를 재사용하는 것이 중요하다. Activation 또한, 한번 로드되면, 좌에서 우로 인접한 PE로 stream되며 reuse 극대화를 통해서 memory로부터 데이터를 불러오는 에너지, 시간을 최소화하며 효율적이고 빠른 연산을 구현할 수 있다. 이를 구현하기 위해서 위와 같이 in_shift_register와 w_shift_register를 활용했다. buffer_active 신호를 통해 각 레지스터에 activation, weight를 불러와 저장할 수 있다.

in_shift_register는 일종의 streaming scheduler로 각 행의 1열에 해당하는 PE에 좌에서 우 방향으로 activation값들이 입력되도록 한다.

이 레지스터에는 64비트 데이터(matrix_buffer: 8bit X 8)가 저장되어 있다. 하위 8비트 값은 pipe_row[0]으로 출력되고(ex.matrix_buffer[7:0]), in_shift가 활성화되면서 그 다음 8비트 에 해당하는 값이(ex.matrix_buffer[15:8]) 하위 8비트가 되는 구조이다. 이 과정이 반복되면서, 각 행의 1열 PE로 미리 설정해놓은 순서로 8개의 activation값이 모두 입력된다. w_shift_register 또한 마찬가지로 filter_buffer에 설정해놓은 순서대로 weight값들이 w_shift 신호에 따라서 각 열의 1행 PE로 입력된다. 이때, weight값은 9개이므로, 각 열당 3개씩 즉, 하나의 w_shift_register에는 24비트 데이터가 buffer_active 신호를 통해 저장된다.

```

PE pe_11(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[0]),
    .sum_active (in_shift[0]),

    .sum_in   (8'd0),
    .matrix_in (pipe_row1[0]),
    .filter_in (pipe_col1[0]),

    .matrix_out (pipe_row1[1]),
    .filter_out (pipe_col1[1]),
    .sum_out   (pipe_sum1[1])
);

PE pe_12(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[0]),
    .sum_active (in_shift[0]),

    .sum_in   (8'd0),
    .matrix_in (pipe_row1[1]),
    .filter_in (pipe_col2[0]),

    .matrix_out (pipe_row1[2]),
    .filter_out (pipe_col2[1]),
    .sum_out   (pipe_sum2[1])
);

PE pe_13(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[0]),
    .sum_active (in_shift[0]),

    .sum_in   (8'd0),
    .matrix_in (pipe_row1[2]),
    .filter_in (pipe_col3[0]),

    .matrix_out (),
    .filter_out (pipe_col3[1]),
    .sum_out   (pipe_sum3[1])
);

PE pe_21(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[1]),
    .sum_active (in_shift[1]),

    .sum_in   (pipe_sum1[1]),
    .matrix_in (pipe_row2[0]),
    .filter_in (pipe_col1[1]),

    .matrix_out (pipe_row2[1]),
    .filter_out (pipe_col1[2]),
    .sum_out   (pipe_sum1[2])
);

PE pe_22(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[1]),
    .sum_active (in_shift[1]),

    .sum_in   (pipe_sum2[1]),
    .matrix_in (pipe_row2[1]),
    .filter_in (pipe_col2[1]),

    .matrix_out (pipe_row2[2]),
    .filter_out (pipe_col2[2]),
    .sum_out   (pipe_sum2[2])
);

PE pe_23(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[1]),
    .sum_active (in_shift[1]),

    .sum_in   (pipe_sum3[1]),
    .matrix_in (pipe_row2[2]),
    .filter_in (pipe_col3[1]),

    .matrix_out (),
    .filter_out (pipe_col3[2]),
    .sum_out   (pipe_sum3[2])
);

PE pe_31(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[2]),
    .sum_active (in_shift[2]),

    .sum_in   (pipe_sum1[2]),
    .matrix_in (pipe_row3[0]),
    .filter_in (pipe_col1[2]),

    .matrix_out (pipe_row3[1]),
    .filter_out (),
    .sum_out   (sum1)
);

PE pe_32(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[2]),
    .sum_active (in_shift[2]),

    .sum_in   (pipe_sum2[2]),
    .matrix_in (pipe_row3[1]),
    .filter_in (pipe_col2[2]),

    .matrix_out (pipe_row3[2]),
    .filter_out (),
    .sum_out   (sum2)
);

PE pe_33(
    .clk      (clk),
    .rst      (rst),
    .filter_active(w_shift),
    .matrix_active(in_shift[2]),
    .sum_active (in_shift[2]),

    .sum_in   (pipe_sum3[2]),
    .matrix_in (pipe_row3[2]),
    .filter_in (pipe_col3[2]),

    .matrix_out (),
    .filter_out (),
    .sum_out   (sum3)
);

```

fig5.12) Systolic Array 3x3 Verilog Code 2

위와 같이 9개의 PE module을 구성하여, 인접한 PE와 activation과 weight 그리고 partial sum을 주고 받을 수 있도록 연결하였다. 각PE는 w_shift와 in_shift 신호를 통해서, PE 내부로 streaming scheduler로부터 출력되는 데이터가 저장되도록 하였다. pipe_col[0] → pipe_col[1] → pipe_col[2] 순서로 weight가 이동하고 각 열의 위에서 아래로 값이 내려오는 방식이다. activation값은 pipe_row[0] → pipe_row[1] → pipe_row[2] 순서로 이동하고, 각 행의 좌에서 우 방향으로 데이터가 이동하는 방식이다. 누적합은 pipe_sum[0] → pipe_sum[1] → sum 순서로 이동하고, 위에서 아래로 값이 내려오는 방식이다. 마지막 행의 각 열의 PE로부터 출력되는 sum1, sum2, sum3을 통해서 결과 행렬의 각 요소에 필요한 누적합들을 조합하여 최종 연산 결과를 도출하도록 하였다. 정리하자면, activation과 weight가 streaming scheduler로 미리 지정한 순서대로 입력되고, 각각 인접한 PE 사이에서 streaming 및 MAC 동작을하여, 마지막 행에서 누적합을 받아오는 방식이다.

```

209 Accumulator acc_c1(.clk(clk), .rst(rst), .active(acc_active[3]), .in(c11), .out(c1));
210 Accumulator acc_c2(.clk(clk), .rst(rst), .active(acc_active[2]), .in(c12), .out(c2));
211 Accumulator acc_c3(.clk(clk), .rst(rst), .active(acc_active[1]), .in(c21), .out(c3));
212 Accumulator acc_c4(.clk(clk), .rst(rst), .active(acc_active[0]), .in(c22), .out(c4));
213
214 assign matrix_buffer1 = {a24,a23,a22,a21,a14,a13,a12,a11};
215 assign matrix_buffer2 = {a34,a33,a32,a31,a24,a23,a22,a21};
216 assign matrix_buffer3 = {a44,a43,a42,a41,a34,a33,a32,a31};
217
218 assign filter_buffer1 = {b33,b23,b13};
219 assign filter_buffer2 = {b32,b22,b12};
220 assign filter_buffer3 = {b31,b21,b11};

```

fig5.13) Systolic Array 3x3 Verilog Code 3

```

241 always @(*) begin
242     next_state = state;
243
244     cnt_rst=1'b0;
245
246     cnt_active=1'b0;
247
248
249     acc_active=4'b0000;
250     buffer_active=1'b0;
251
252     in_shift=3'b000;
253     w_shift=1'b0;
254
255     sys33_done=1'b0;
256
257     c11=8'd0;
258     c12=8'd0;
259     c21=8'd0;
260     c22=8'd0;
261
262 case (state)
263     S0: begin
264         if (sys33_active)
265             next_state = S1;
266         end
267
268     S1: begin
269         buffer_active = 1'b1;
270         cnt_rst = 1'b1;
271         next_state = S2;
272         end
273
274     S2: begin
275         cnt_active = 1'b1;
276         w_shift = 1'b1;
277         if(cnt == 2) begin
278             next_state = S3;
279         end
280         end
281
282     S3: begin
283         in_shift = 3'b001;
284         cnt_rst = 1'b1;
285         next_state = S4;
286         end
287
288     S4: begin
289         in_shift = 3'b011;
290         next_state = S5;
291         end
292
293     S5: begin
294         in_shift = 3'b111;
295         next_state = S6;
296         end
297
298     S6: begin
299         in_shift = 3'b111;
300         acc_active = 4'b1000;
301         c11 = sum1;
302         next_state = S7;
303         end
304
305     S7: begin
306         in_shift = 3'b111;
307         acc_active = 4'b0100;
308         c12 = sum1;
309         next_state = S8;
310         end
311
312     S8: begin
313         in_shift = 3'b111;
314         acc_active = 4'b1000;
315         c11 = sum2;
316         next_state = S9;
317         end
318
319     S9: begin
320         in_shift = 3'b111;
321         acc_active = 4'b0100;
322         c12 = sum2;
323         next_state = S10;
324         end
325
326     S10: begin
327         in_shift = 3'b111;
328         acc_active = 4'b0010;
329         c21 = sum3;
330         next_state = S11;
331         end
332
333     S11: begin
334         in_shift = 3'b111;
335         acc_active = 4'b0010;
336         c21 = sum3;
337         next_state = S12;
338         end
339
340     S12: begin
341         in_shift = 3'b111;
342         acc_active = 4'b0010;
343         c21 = sum3;
344         next_state = S13;
345         end
346
347     S13: begin
348         in_shift = 3'b111;
349         acc_active = 4'b0010;
350         c21 = sum3;
351         next_state = S14;
352         end
353
354     S14: begin
355         in_shift = 3'b111;
356         acc_active = 4'b0010;
357         c21 = sum3;
358         next_state = S15;
359         end
360
361     S15: begin
362         in_shift = 3'b111;
363         acc_active = 4'b0010;
364         c21 = sum3;
365         next_state = S16;
366         end
367
368     S16: begin
369         sys33_done = 1'b1;
370         next_state = S0;
371         end
372 endcase
373 endmodule

```

fig5.14) Systolic Array 3x3 Verilog Code 4, 5, 6

무어(Moore) 머신 방식으로 FSM을 구현하였다. clock counter를 통해서 filter_buffer의 값들을 모두 각 PE에 내부에 저장 및 고정시켜 WS(Weight Stationary)방식을 구현하였다. 그 후, 순차적으로 matrix_buffer의 값들을 streaming 시켰다. 마지막 행의 PE에 누적합이 출력되면, acc_active 신호를 통해 해당 값이 필요한 결과 행렬 요소의 accumulator로 값이 누적된다.

```

303 end
304
305 S7: begin
306     in_shift = 3'b111;
307     acc_active = 4'b0100;
308     c12 = sum1;
309     next_state = S8;
310 end
311
312 S8: begin
313     in_shift = 3'b111;
314     acc_active = 4'b1000;
315     c11 = sum2;
316     next_state = S9;
317 end
318
319 S9: begin
320     in_shift = 3'b111;
321     acc_active = 4'b0100;
322     c12 = sum2;
323     next_state = S10;
324 end
325
326 S10: begin
327     in_shift = 3'b111;
328     acc_active = 4'b0010;
329     c21 = sum3;
330     next_state = S11;
331 end
332
333 S11: begin
334     in_shift = 3'b111;
335     acc_active = 4'b0010;
336     c21 = sum3;
337     next_state = S12;
338 end
339
340 S12: begin
341     in_shift = 3'b111;
342     acc_active = 4'b0010;
343     c21 = sum3;
344     next_state = S13;
345 end
346
347 S13: begin
348     in_shift = 3'b111;
349     acc_active = 4'b0010;
350     c21 = sum3;
351     next_state = S14;
352 end
353
354 S14: begin
355     in_shift = 3'b111;
356     acc_active = 4'b0010;
357     c21 = sum3;
358     next_state = S15;
359 end
360
361 S15: begin
362     in_shift = 3'b111;
363     acc_active = 4'b0010;
364     c21 = sum3;
365     next_state = S16;
366 end
367
368 S16: begin
369     sys33_done = 1'b1;
370     next_state = S0;
371 end
372 endcase
373 endmodule

```

fig5.15) Systolic Array 3x3 Verilog Code 7, 8

모든 연산 과정이 끝나면 sys33_done 신호를 통해서 computation module과 controller에 3 x 3연산이 종료되었음을 알린다.

b. simulation result

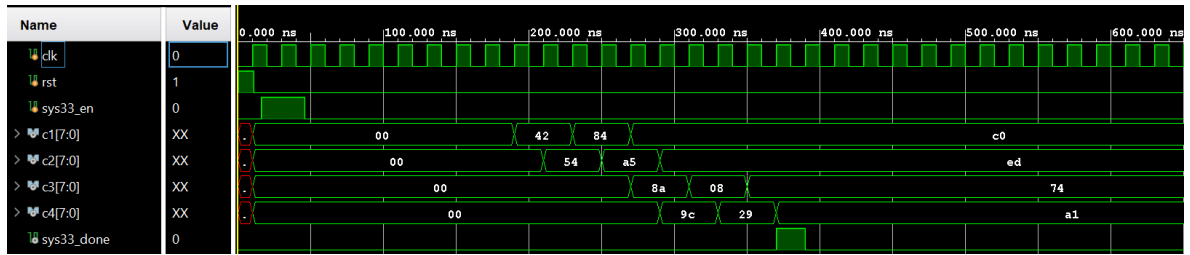


fig5.16) Systolic Array 3x3 Testbench Waveform

Single PE의 테스트 벤치와 같은 input matrix와 filter matrix를 투입하여 3 x 3 systolic array를 테스트하였다. sys33_active 신호가 활성화 되면서 연산이 시작되고, 각 결과 행렬 요소에 누적합이 진행되는 것을 확인할 수 있다. 최종 연산 결과 또한 Single PE의 연산 결과와 일치하며 정확하게 계산된 것을 확인할 수 있다. 마지막 c22에 해당하는 값이 완성되면서, 연산이 종료되었음을 알리는 sys33_done 신호 또한 정상적으로 활성화 되는 것을 확인할 수 있다.

5.4) Systolic Array 2x2

a. code

```

3 module sys_22(
4     input clk, rst, sys22_active,
5
6     input [7:0] a11, a12, a13, a14,
7         a21, a22, a23, a24,
8         a31, a32, a33, a34,
9         a41, a42, a43, a44,
10
11     input [7:0] b11, b12, b13,
12         b21, b22, b23,
13         b31, b32, b33,
14
15     output [7:0] c1, c2, c3, c4,
16
17     output reg [4:0] state,
18     output reg sys22_done
19 );
20
21 reg [4:0] next_state;
22
23 reg [7:0] matrix_in_top; // data_in1
24 reg [7:0] matrix_in_bottom; // data_in2
25 reg [7:0] filter_in_left; // filter_in1
26 reg [7:0] filter_in_right;
27
28 wire [7:0] pe1_sum, pe2_sum, pe3_sum, pe4_sum;
29 wire [7:0] matrix_pipe_top, matrix_pipe_bottom;
30 wire [7:0] filter_pipe_left, filter_pipe_right;
31
32 reg [7:0] acc_save1, acc_save2, acc_save3, acc_save4;
33 reg acc_active1, acc_active2, acc_active3, acc_active4;
34 wire [7:0] w_c1, w_c2, w_c3, w_c4;
35
36 reg matrix_active;
37 reg filter_active;

```

fig5.17) Systolic Array 2x2 Verilog Code 1, 2

Systolic array 2 x 2를 구현하기 위해서 위와 같이 변수를 설정하였다. 전체적인 방식은 systolic array 3x3방식과 유사하다. 우선 systolic array 2x2는 PE 4개를 2행 2열로 배치하는 것으로 시작한다. 따라서 matrix_in_top, bottom을 통해서 activation 값들이 두개의 행의 1열에 해당하는 PE로 입력된다. filter_in_left와 filter_in_right를 통해서 weight값 또한 1행의 두개의 열에 위치한 PE로 입력된다. pe_sum은 4개의 PE에서 계산된 누적합을 의미하며, matrix_active와 filter_active를 통해서 activation과 weight값들이 인접한 PE로 streaming 된다. pe_sum 각각의 PE에서 계산된 누적합을 각 결과 행렬 요소를 만드는 accumulator로 전달하며, acc_active 신호를 보내서 accumulator가 입력받은 값을 누적하도록 하였다.

```

106 Accumulator acc1 (
107   .clk (clk),
108   .rst (rst),
109   .active (acc_active1),
110   .in (acc_save1),
111   .out (w_c1)
112 );
113
114 Accumulator acc2 (
115   .clk (clk),
116   .rst (rst),
117   .active (acc_active2),
118   .in (acc_save2),
119   .out (w_c2)
120 );
121
122 Accumulator acc3 (
123   .clk (clk),
124   .rst (rst),
125   .active (acc_active3),
126   .in (acc_save3),
127   .out (w_c3)
128 );
129
130 Accumulator acc4 (
131   .clk (clk),
132   .rst (rst),
133   .active (acc_active4),
134   .in (acc_save4),
135   .out (w_c4)
136 );
137
138 // (0,0) PE : top-left
139 PE pe1 (
140   .clk (clk),
141   .rst (rst),
142   .filter_active (filter_active),
143   .matrix_active (matrix_active),
144   .sum_active (matrix_active),
145   .sum_in (8'd0),
146   .matrix_in (matrix_in_top),
147   .filter_in (filter_in_left),
148   .matrix_out (matrix_pipe_top),
149   .filter_out (filter_pipe_left),
150   .sum_out (pe1_sum)
151 );
152
153 // (0,1) PE : top-right
154 PE pe2 (
155   .clk (clk),
156   .rst (rst),
157   .filter_active (filter_active),
158   .matrix_active (matrix_active),
159   .sum_active (matrix_active),
160   .sum_in (8'd0),
161   .matrix_in (matrix_pipe_top),
162   .filter_in (filter_in_right),
163   .matrix_out (),
164   .filter_out (filter_pipe_right),
165   .sum_out (pe2_sum)
166 );
167
168 // (1,0) PE : bottom-left
169 PE pe3 (
170   .clk (clk),
171   .rst (rst),
172   .filter_active (filter_active),
173   .matrix_active (matrix_active),
174   .sum_active (matrix_active),
175   .sum_in (pe1_sum),
176   .matrix_in (matrix_in_bottom),
177   .filter_in (filter_pipe_left),
178   .matrix_out (matrix_pipe_bottom),
179   .filter_out (),
180   .sum_out (pe3_sum)
181 );
182
183 // (1,1) PE : bottom-right
184 PE pe4 (
185   .clk (clk),
186   .rst (rst),
187   .filter_active (filter_active),
188   .matrix_active (matrix_active),
189   .sum_active (matrix_active),
190   .sum_in (pe2_sum),
191   .matrix_in (matrix_pipe_bottom),
192   .filter_in (filter_pipe_right),
193   .matrix_out (),
194   .filter_out (),
195   .sum_out (pe4_sum)
196 );

```

fig5.18) Systolic Array 2x2 Verilog Code 3, 4

위와 같이 PE module 4개를 구성하여 weight와 activation 그리고 partial sum을 인접한 PE로 주고 받을 수 있도록 연결하였다. matrix_in, filter_in을 통해서 PE에 data flow가 들어오도록 하였다. 또한 matrix_pipe를 통해서 activation 값이 두 행에서 좌에서 우 방향으로 stream되도록 하였고, filter_pipe를 통해서 weight 값이 위에서 아래로 stream되도록 하였다. 1행 PE들이 누적인 값들은 2행으로 전달되고, systolic array 3x3 방식과는 다르게 모든 PE의 누적합을 accumulator에서 결과 행렬 계산을 위해 조합하도록 하였다.

```

139 always @(posedge clk or posedge rst) begin
140   if (rst)
141     state <= S0;
142   else
143     state <= next_state;
144   end
145
146 // Next-State * Control Logic
147 always @(*) begin
148   next_state = state;
149   sys22_done = 1'b0;
150
151   matrix_in_top = 8'd0;
152   matrix_in_bottom = 8'd0;
153   filter_in_left = 8'd0;
154   filter_in_right = 8'd0;
155
156   acc_active1 = 1'b0;
157   acc_active2 = 1'b0;
158
159   acc_save1 = pe3_sum;
160   next_state = S6;
161 end
162
163 S6: begin
164   matrix_in_top = a21;
165   matrix_in_bottom = a23;
166   acc_active2 = 1'b1;
167   acc_save2 = pe1_sum;
168   next_state = S7;
169 end
170
171 S7: begin
172   matrix_in_top = a22;
173   matrix_in_bottom = a31;
174   acc_active1 = 1'b1;
175   acc_save1 = pe4_sum;
176   next_state = S8;
177 end
178
179 S8: begin
180   filter_in_left = b23;
181   filter_in_right = b22;
182   filter_active = 1'b1;
183   matrix_active = 1'b0;
184   next_state = S2;
185 end
186
187 S2: begin
188   filter_in_left = b33;
189   filter_in_right = b32;
190   filter_active = 1'b1;
191   matrix_active = 1'b0;
192   next_state = S3;
193 end
194
195 S3: begin
196   matrix_in_top = 8'd0;
197   matrix_in_bottom = 8'd0;
198   acc_active3 = 1'b1;
199   acc_save3 = pe4_sum;
200   next_state = S11;
201 end
202
203 // 2nd Filter
204 S11: begin
205   filter_in_left = b12;
206   filter_in_right = b11;
207   filter_active = 1'b1;
208   matrix_active = 1'b0;
209   acc_active4 = 1'b1;
210   acc_save4 = pe1_sum;
211   next_state = S12;
212 end
213
214 S12: begin
215   matrix_in_top = 8'd0;
216   matrix_in_bottom = 8'd0;
217   acc_active4 = 1'b1;
218   acc_save4 = pe3_sum;
219   next_state = S22;
220 end
221
222 S22: begin
223   matrix_in_top = 8'd0;
224   matrix_in_bottom = 8'd0;
225   sys22_done = 1'b1;
226   next_state = S0;
227 end
228
229 endmodule

```

fig5.19) Systolic Array 2x2 Verilog Code 5, 6, 7

Systolic array 2x2 또한 무어(Moore) 방식 FSM으로 구현하였다. 앞서 설명했듯이, 3x3 filter는 2x2 사이즈를 초과하기 때문에 Weight Stationary 방식을 구현하려면 phase별로 각 PE에 weight를 바꿔가며 공급해야 한다. 이를 구현하기 위해 3개의 phase로 나뉘었으며, 각 phase마다 weight를 4개의 PE에 저장 및 고정시키는 부분과 activation 값을 streaming하는 부분으로 구성하였다. 각 결과 행렬 요소를 만들기 위해 필요한 누적합이 PE의 sum으로 출력되면 sum값을 해당 결과 행렬 요소를 만드는 accumulator에 입력하고 active 신호를 주어 누적하도록 하였다. 그렇게 첫번째 phase가 끝나면 다음 phase로 넘어가기 위해 weight를 각 PE에 재공급한다. 새로운 weight를 입력하고 filter_active 신호를 활성화하여 weight를 고정한다. 이 과정이 반복되면서 3개의 phase를 거치며 각 결과행렬 요소를 완성하도록 하였다. 모든 과정이 끝나면 sys22_done 신호를 활성화하여 연산이 끝났음을 controller와 computation module에 전달한다. sys22_done신호가 controller에 전달되면 다음 과정인 display module이 활성화된다.

b. simulation result

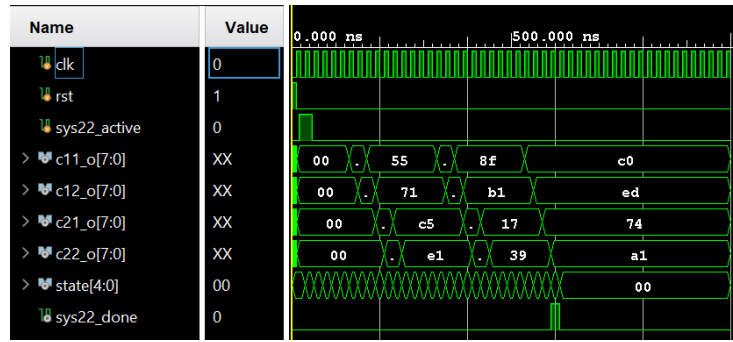


fig5.20) Systolic Array 2x2 Testbench Waveform

지금까지 연산 방식들과 마찬가지로 같은 input matrix와 filter matrix를 투입하여 systolic 2x2 연산을 확인하였다. sys22_active 신호가 활성화 되면서 연산이 시작되고, 각 결과 행렬 요소들이 누적되면서 완성되어가는 것을 확인할 수 있다. 최종 결과는 이전까지의 연산 결과와 일치하며 정확하게 계산되는 것을 확인할 수 있다. 모든 연산이 종료되면 sys22_done신호를 정상적으로 출력하는 것을 볼 수 있다.

5.5) Controller

a. code

```

`timescale 1ns / 1ps

module Controller(
    input active,
    input clk, rst,
    input mem_done, pe_done, sys33_done, sys22_done, display_done,

    output mem_active, pe_active, sys33_active, sys22_active, display_active,
    output process_inactive
);

    reg [3:0] state, next_state;

```

fig5.21) Controller Verilog Code 1

Controller module을 구현하기 위해서 위와 같이 변수를 설정하였다 active, clk, rst는 FSM의 시작 조건을 제공하며, mem_done, pe_done, sys33_done, sys22_done, display_done은 각 연산 모듈의 완료 여부를 알려주며 다음 상태로의 이동을 결정한다. 출력 신호인 mem_active, pe_active, sys33_active, sys22_active, display_active는 해당하는 연산 단계에서 모듈을 활성화하는 신호이며, process_inactive는 전체 연산 과정이 종료되었음을 나타낸다. 마지막으로 state와 next_state 는 FSM의 현재 상태와 다음 상태를 저장하여 전체 연산의 흐름이 순서대로 작동하도록 제어하는 변수로 사용된다.

```

16 always @(posedge clk)
17 begin
18     if(rst) state <= S0;
19     else state <= next_state;
20 end
21
22 always @(*)
23 begin
24     next_state = state;
25     case(state)
26         S0: next_state = (active) ? S1 : S0;
27         S1: next_state = (mem_done) ? S2 : S1;
28         S2: next_state = (pe_done) ? S3 : S2;
29         S3: next_state = (sys33_done) ? S4 : S3;
30         S4: next_state = (sys22_done) ? S5 : S4;
31         S5: next_state = (display_done) ? S6 : S5;
32         S6: next_state = S0;
33         default: next_state = S0;
34     endcase
35 end
37 assign mem_active = (state == S1);
38 assign pe_active = (state == S2);
39 assign sys33_active = (state == S3);
40 assign sys22_active = (state == S4);
41 assign display_active = (state == S5);
42 assign process_inactive = (state == S6);

```

fig5.22) Controller Verilog Code 2, 3

State 위와 같이 총 7개의 상태(S0~S6)로 구성되며, S0에서 active 신호를 대기하다가 활성화되면 S1로 전이하여 mem_active 신호를 통해 Memory 모듈을 작동시키고, mem_done이 입력되면 S2로 넘어가 pe_active를 통해 PE 모듈 연산을 수행한다. 이후 pe_done 입력이 들어오면 S3으로 이동해 sys33_active 신호로 3×3 systolic array 연산을 진행하며, 연산 종료(sys33_done) 시 S4에서 sys22_active 신호를 발생시켜 2×2 systolic 연산을 실행하고, sys22_done 신호가 들어오면 S5로 넘어가 display_active를 통해 Display Module이 최종 결과를 출력하도록 한다. 디스플레이가 완료되면(display_done) S6 상태로 전환되어 process_inactive 신호를 통해 전체 연산 종료를 선언하고, 이후 자동으로 S0으로 복귀해 새로운 active 신호를 대기하도록 설계하였다.

b. simulation result



fig5.23) Controller Testbench Waveform

시뮬레이션 결과, Controller는 설계된 순서대로 상태를 전이하며 각 단계별 active 신호가 정확한 시점에만 활성화됨이 확인된다. active 입력 후 mem_active → pe_active → sys33_active → sys22_active → display_active 순으로 출력이 정상적으로 진행되었고, 각 단계의 done 신호에 따라 다음 상태로 안정적으로 넘어갔다. 마지막에는 process_inactive가 활성화되며 전체 연산 과정이 종료되었음이 표시되고, 해당 모듈이 정상적으로 동작함이 확인되었다.

5.6) Memory

a. code

```

3 module Memory(
4     input clk, rst, mem_active, shutdown,
5     input s_out_en,
6
7     input s33_out_en, s22_out_en,
8
9     input [7:0] a11, a12, a13, a14,
10    a21, a22, a23, a24,
11    a31, a32, a33, a34,
12    a41, a42, a43, a44,
13    output [7:0] a11_out, a12_out, a13_out, a14_out,
14    a21_out, a22_out, a23_out, a24_out,
15    a31_out, a32_out, a33_out, a34_out,
16    a41_out, a42_out, a43_out, a44_out,
17
18    input [7:0] b11, b12, b13,
19    b21, b22, b23,
20    b31, b32, b33,
21    output [7:0] b11_out, b12_out, b13_out,
22    b21_out, b22_out, b23_out,
23    b31_out, b32_out, b33_out,
24
25    input [7:0] s_c11, s_c12, s_c21, s_c22,
26    output [7:0] s_c11_out, s_c12_out, s_c21_out, s_c22_out,
27
28    input [7:0] s33_c11, s33_c12, s33_c21, s33_c22,
29    output [7:0] s33_c11_out, s33_c12_out, s33_c21_out, s33_c22_out,
30
31    input [7:0] s22_c11, s22_c12, s22_c21, s22_c22,
32    output [7:0] s22_c11_out, s22_c12_out, s22_c21_out, s22_c22_out,
33
34    input [3:0] a_reg_num, b_reg_num,
35    output reg [7:0] a_reg_out, b_reg_out,
36
37    output reg mem_done
38 );
39
40 wire rst_global;
41
42 or_gate set_reset (.a(rst), .b(shutdown), .out(rst_global));

```

fig5.24) Memory Verilog Code 1, 2

Memory 모듈은 Input Matrix, Weight Matrix, 그리고 연산 결과값들을 저장하고 필요한 시점에 출력하는 버퍼 역할을 수행한다. 이를 위해 clk, rst, shutdown은 레지스터 저장값의 초기화와 동작 동기화를 담당하고, mem_active-s_out_en-s33_out_en-s22_out_en 신호는 각 데이터의 저장을 제어한다. 또한 a_reg_num과 b_reg_num은 A-B 행렬에서 원하는 값을 선택하기 위한 신호이며, a_reg_out과 b_reg_out은 이에 대응하는 조합논리 기반 MUX 출력이다. 마지막으로 mem_done은 필요한 메모리 로딩이 완료되었음을 알리는 신호로, mem_active와 함께 자동으로 실행된다.

```

72 | Register s1_c11_reg (.clk(clk), .rst(rst_global), .active(s_out_en), .in(s_c11), .out(s_c11_out));
73 | Register s1_c12_reg (.clk(clk), .rst(rst_global), .active(s_out_en), .in(s_c12), .out(s_c12_out));
74 | Register s1_c21_reg (.clk(clk), .rst(rst_global), .active(s_out_en), .in(s_c21), .out(s_c21_out));
75 | Register s1_c22_reg (.clk(clk), .rst(rst_global), .active(s_out_en), .in(s_c22), .out(s_c22_out));
76 |

```

fig5.25) Memory Verilog Code 3

복수의 Register 인스턴스로 입력 데이터를 저장하고 필요한 시점에 출력하는 구조로 구성하였다. 전체 리셋 신호는 rst와 shutdown을 OR 연산하여 생성한 rst_global 신호로 통합하였으며, 여기서 shutdown은 process_inactive 상태, 즉 전체 연산 프로세스가 종료되었을 때 Controller에서 발생시키는 전역 리셋 신호를 의미한다. 이를 통해 모든 레지스터의 초기화가 동시에 일어나도록 하였다.

Input Matrix(a11~a44)와 Weight Matrix(b11~b33)는 mem_active가 1일 때 저장되어 연산 과정 동안 유지된다. 결과값 레지스터(s_cXX, s33_cXX, s22_cXX) 역시 각 enable 신호에 따라 알맞게 갱신되도록 하였다. 또한 a_reg_num, b_reg_num을 이용한 MUX 구조는 A-B 행렬 중 원하는 요소를 선택해 systolic array가 요구하는 순서대로 스트리밍할 수 있도록 하며, 이는 3×3과 2×2 systolic 방식 모두의 데이터 흐름에 대응된다. 마지막으로 mem_done은 mem_active에 연동되어 메모리 로딩이 완료되었음을 Controller에 알리는 완료 신호이다.

b. simulation result

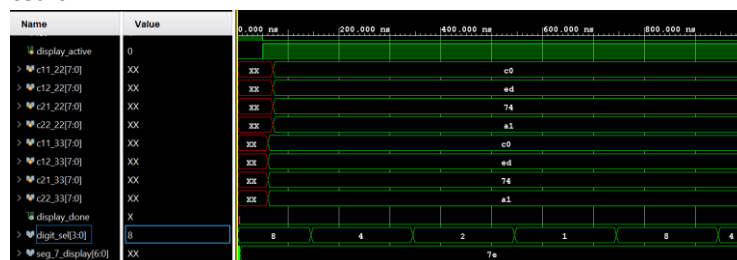


fig5.26) Memory Testbench Waveform

시뮬레이션 wave를 확인한 결과, mem_active 신호가 활성화되는 구간에서 Input Matrix a11~a44 값이 Register에 정확하게 입력되며, 각 출력(a11_out~a44_out)이 해당 시점 이후 안정적으로 유지되는 것을 확인하였다. 또한 s_out_en, s33_out_en, s22_out_en 신호가 각각 활성화될 때 대응하는 결과 레지스터가 정상적으로 갱신되며, enable 신호가 비활성화된 구간에서는 모든 출력이 변화 없이 유지되어 메모리 버퍼 구조가 의도한 대로 동작함을 보여준다. 특히 입력 데이터가 순차적으로 저장된 후 지속적으로 동일한 값을 유지하는 모습은 Memory 모듈이 systolic array나 PE 연산 과정에서의 데이터 보존 기능을 제대로 수행되고 있음을 나타낸다.

5.7) Display

a. code

```

39  always @(posedge clk) begin
40      if (rst) begin
41          sec_cnt <= 0;
42      end
43
44      else if (sec_cnt==9) begin
45          sec_cnt <= 0;
46      end
47
48      //else if(sec_clk_cnt==27'd99) begin // for tb
49      else if(sec_clk_cnt==27'd9999999) begin
50
51          sec_cnt<=sec_cnt+1;
52      end
53  end
54
55  always @(posedge clk) begin
56      if (rst) begin
57          seg_clk_cnt <= 0;
58          seg_clk<=0;
59      end
60
61
62      //else if (seg_clk_cnt==27'd9) begin // for tb
63      else if (seg_clk_cnt==27'd99999) begin
64          seg_clk_cnt <= 0;
65          seg_clk=~seg_clk;
66      end
67
68      else if(display_active) begin
69
70          seg_clk_cnt<=seg_clk_cnt+1;
71      end
72
73  end

```

fig5.27) Display Verilog Code 1, 2

본 모듈은 고속 시스템 클록을 직접 사용하지 않고, 내부에서 두 종류의 분주 클록을 생성한다. 먼저 sec_clk_cnt는 일정 카운트에 도달할 때 하나의 전환 타이밍 펄스를 만드는 역할을 하며, 이를 이용해 출력해야 할 결과값을 일정 주기(sec_cnt)에 맞추어 순차적으로 바꾼다. 또한 seg_clk_cnt는 7-Segment 멀티플렉싱을 위한 느린 주기의 클럭 seg_clk을 생성하여, 사람이 인식할 수 있는 주기로 각 자리수를 번갈아 표시할 수 있게 한다. 이 두 분주 로직은 display_active 상태에서만 동작하며, rst에 의해 초기화된다.

```

22  always @(posedge clk) begin
23      if (rst) begin
24          sec_clk_cnt <= 0;
25      end
26
27      //else if (seg_clk_cnt==27'd99) begin //for tb
28      else if (sec_clk_cnt==27'd9999999) begin
29          sec_clk_cnt <= 0;
30
31      end
32      else if(display_active) begin
33
34          sec_clk_cnt<=sec_clk_cnt+1;
35      end
36
37  end
38
111  always @(*) begin
112      case(sec_cnt)
113          4'd1 : c_num = c11_33;
114          4'd2 : c_num = c12_33;
115          4'd3 : c_num = c21_33;
116          4'd4 : c_num = c22_33;
117          4'd5 : c_num = c11_22;
118          4'd6 : c_num = c12_22;
119          4'd7 : c_num = c21_22;
120          4'd8 : c_num = c22_22;
121          default: c_num = 0;
122      endcase
123  end

```

fig5.28) Display Verilog Code 3, 4

sec_cnt는 sec_clk_cnt에 따라 증가하며 현재 화면에 표시할 결과 번호를 결정한다. 1~4는 3x3 systolic 결과를, 5~8은 2x2 systolic의 결과를 선택하여 c_num에 전달한다. 이를 통해 총 8개의 결과가 일정 주기 간격으로 순차적으로 표시되도록 설계하였다.

```

75  always @(posedge seg_clk, posedge rst) begin
76      if (rst) begin
77          digit <= 4'b1000;
78      end
79
80      else if (display_active) begin
81          digit <= {digit[0], digit[3:1]};
82      end
83  end
84
85  always @(*) begin
86      case(digit)
87          4'b1000: LED_BCD=c_num / 1000;
88          4'b0100: LED_BCD=(c_num%1000) / 100;
89          4'b0010: LED_BCD=((c_num%1000)%100) / 10;
90          4'b0001: LED_BCD=((c_num%1000)%100) % 10;
91          default: LED_BCD=0;
92      endcase
93  end
94
95  always @(*) begin
96      case(LED_BCD)
97          4'b0000: seg7 = 7'b1111110; //0
98          4'b0001: seg7 = 7'b0110000; //1
99          4'b0010: seg7 = 7'b1101101; //2
100         4'b0011: seg7 = 7'b1111001; //3
101         4'b0100: seg7 = 7'b0110011; //4
102         4'b0101: seg7 = 7'b1011011; //5
103         4'b0110: seg7 = 7'b1011111; //6
104         4'b0111: seg7 = 7'b1110000; //7
105         4'b1000: seg7 = 7'b1111111; //8
106         4'b1001: seg7 = 7'b1111011; //9
107         default: seg7 = 7'b1111110; //0
108     endcase
109  end

```

fig5.29) Display Verilog Code 5, 6

Display 모듈은 해당하는 연산 결과값(c_num)을 7-Segment에 출력하기 위해 먼저 해당 값을 각각의 자리수로 분해하는 BCD 변환 과정을 수행한다. digit 신호가 가리키는 자리(1000-0100-0010-0001)에 따라, c_num을 정수 나눗셈과 나머지 연산을 이용하여 해당 자리수의 BCD 값(0~9)을 LED_BCD에 저장한다. 이후 LED_BCD는 7-Segment 디코더에 전달되어 숫자 0~9에 대응하는 세그먼트 패턴(seg7)으로 변환된다. 이 디코딩 로직은 각 자리수에 대해 적절한 세그먼트(a~g)를 활성화하여 실제 하드웨어 7-Segment 화면에 사람이 읽을 수 있는 숫자로 출력되도록 하였다. 즉, BCD 분해와 디코딩 과정이 연속적으로 수행되어 c_num의 네 자리 십진수 표현이 빠른 멀티플렉싱 방식으로 표시되며, 모든 과정이 완료되면 display_done 신호를 통해 시스템에 표시 완료 상태를 전달하도록 설계하였다.

b. simulation result

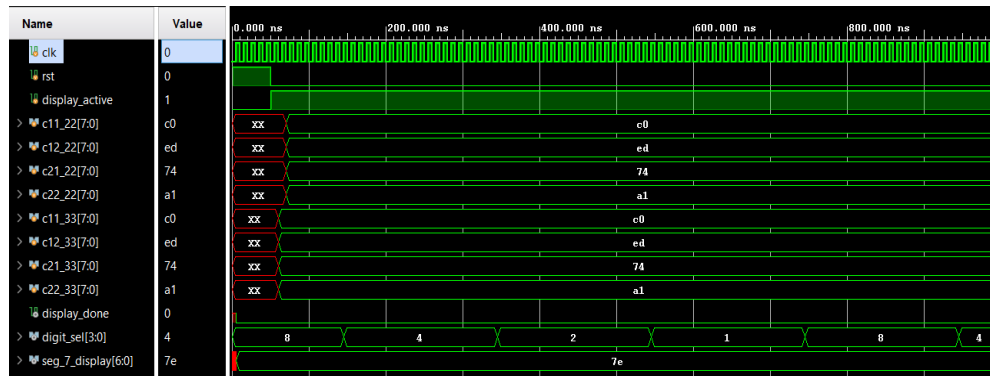


fig5.30) Display Testbench Waveform

시뮬레이션 wave를 분석한 결과, display_active 신호가 활성화된 후 모듈 내부에서 선택된 연산 결과값(c11_22, c12_22)이 정상적으로 c_num에 표시되며, digit_sel 신호가 8 → 4 → 2 → 1 순으로 순환하며 7-Segment의 각 자리를 번갈아 활성화하는 동작이 정확히 수행됨을 확인하였다. 또한 seg_7_display 값이 digit_sel의 변화에 맞추어 해당 자리의 BCD 패턴으로 안정적으로 적용되고, 결과값이 올바른 순서로 표시되는 패턴이 나타났다. 이로서 BCD, 7-Segment 디코딩, 자리 선택 과정이 의도한 대로 작동하고 있음을 보여주며, 전체 Display Module이 정상적으로 동작함을 확인하였다.

5.8) Wrapper

a. code

Wrapper 모듈은 기존 Top 모듈을 바로 해당 FPGA 보드에 implementation하기에는, 입출력 포트 수 과다 등 문제로 오류를 발생하는데, 이를 해결하기 위해 재구성한 상위 모듈이다.

```

41 |
42 | assign rst=~rstn;
43 | assign active=switch;
44 |
45 | assign a11=8'd1; assign a12=8'd2; assign a13=8'd3; assign a14=8'd4;
46 | assign a21=8'd5; assign a22=8'd6; assign a23=8'd7; assign a24=8'd8;
47 | assign a31=8'd9; assign a32=8'd10; assign a33=8'd11; assign a34=8'd12;
48 | assign a41=8'd13; assign a42=8'd14; assign a43=8'd15; assign a44=8'd16;
49 |
50 | assign b11=8'd1; assign b12=8'd2; assign b13=8'd3;
51 | assign b21=8'd4; assign b22=8'd5; assign b23=8'd6;
52 | assign b31=8'd7; assign b32=8'd8; assign b33=8'd9;
53 |
54 | Controller controller_module(
55 | .active(active),
56 | .clk(clk),
57 | .rst(rst),

```

fig5.31)Wrapper Verilog Code

이를 해결하기 위해 Wrapper에서는 모든 input 및 weight 값을 내부에서 할당하여 입력 포트가 없음에도 Top모듈을 구동시킬 수 있도록 하였다. 또한 FPGA 보드의 reset과 switch 입력을 내부 입력 신호로 변환하였다. 내부에서 input 및 weight를 할당하는 부분 외에는 모두 Top모듈의 구조와 동일하고 같은 방식으로 동작한다.

5.9) Computation Time Comparison

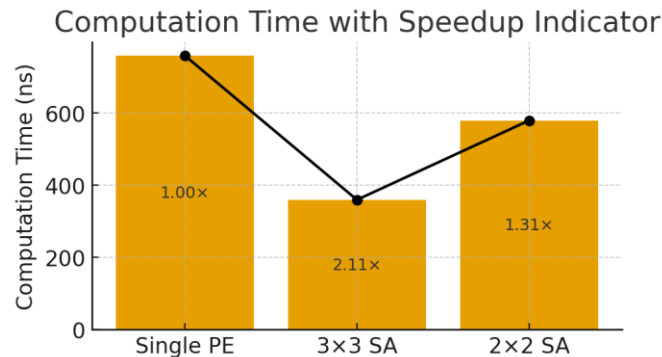


fig5.32) computation time comparison

본 프로젝트에서 각각의 연산 모듈에서 en 신호의 입력과 done 신호의 출력 간의 시간을 기준으로 측정된 연산 지연 시간은 Single PE가 760 ns로 가장 길고, 2x2 SA가 580 ns, 3x3 SA가 360 ns로 측정되었다. Single PE 대비 성능 향상 배수를 비교하면, 2x2 SA는 약 1.31배 속도가 향상되었으며(지연 약 24% 감소), 3x3 SA는 약 2.11배 속도가 향상되었다(지연 약 53% 감소). 이러한 속도 향상 경향은 Chen et al., "Efficient Processing of Deep Neural Networks – Hardware for DNN Processing: Systolic Arrays"에서 분석한 systolic array의 구조적 이점을 잘 반영한다.

먼저 Single PE 구조는 논문에서 "non-systolic sequential datapath"로 분류되는 구조로, 모든 연산이 순차적으로 이루어지며 PE 간 데이터 전달이 존재하지 않는다. 따라서 temporal reuse, spatial reuse, local forwarding 같은 systolic array의 핵심 장점이 전혀 활용되지 못한다. 이로 인해 MAC 연산마다 새로운 ifmap과 weight를 외부 메모리에서 반복적으로 가져와야 하며, 그 과정에서 global memory traffic이 최대화되고 연산 사이클 사이에 bubble이 지속적으로 발생한다. 그 결과 전체 연산 시간이 760 ns로 가장 크게 나타났다.

다음으로 2x2 SA 구조는 systolic 구조의 장점을 부분적으로 활용할 수 있어 Single PE 대비 연산 효율이 크게 향상된다. Chen et al.은 array 크기가 증가할수록 pipeline fill latency가 전체 지연에 미치는 영향이 줄어든다고 설명하는데, 2x2 SA는 4개의 PE가 행·열 방향으로 데이터를 local forwarding하며 Single PE 대비 temporal/spatial reuse가 가능하다. 그러나 array 크기가 작아 pipeline depth가 충분히 확보되지 못하고, dataflow가 일찍 종료되기 때문에 reuse 가능한 연산 횟수 역시 제한적이다. 이러한 이유로 2x2 SA는 Single PE보다 빠르지만(580 ns), 여전히 3x3 SA에 비해서는 느리다.

마지막으로 3x3 SA 구조는 9개의 PE가 완전한 2차원 메시를 이루며 systolic array의 dataflow 최적화 기법을 가장 효과적으로 활용한다. 논문에서는 이를 통해 external DRAM/L2 접근을 크게 줄이고, weight-stationary 방식으로 weight를 열(column) 방향에 고정시켜 temporal reuse window를 크게 확장한다고 설명한다. 또한 pipeline이 채워진 후에는 9개의 모든 PE가 매 사이클마다 유효 연산을 수행하며 steady-state throughput이 최대치에 도달한다. 이러한 구조적 최적화 덕분에 3x3 SA는 전체 구조 중 가장 짧은 360 ns의 연산 시간을 기록하였다.

종합적으로, 본 실험은 논문에서 제시된 분석 즉, array 규모가 커질수록 pipeline 채움 overhead는 줄어들고, data reuse 효율은 증가하며, global memory 접근은 감소한다는 실험적으로 뒷받침한다. 이는 systolic array의 구조적 확장성이 실제로 연산 속도 향상으로 이어짐을 명확하게 보여준다.

6. Result Analysis



fig 6.1) c11, c12, c21, c22 display output

본 실험에서는 4×4 크기의 입력 Feature Map(IFMAP)에 1부터 16까지의 정수를 행 기준으로 배치하고, 3×3 크기의 필터(weight)에는 1부터 9까지의 값을 동일하게 행 기준으로 구성하였다. Convolution 연산은 수학적 정의에 따라 커널을 180° 회전한 후 적용하였으며, stride는 1, padding은 0(valid convolution) 조건을 사용하였다. 이러한 조건에서는 4×4 입력과 3×3 필터의 연산 결과가 2×2 출력 Feature Map으로 산출된다.

먼저, 좌측 상단 출력 c11은 입력의 첫 번째 3×3 영역 (1, 2, 3 / 5, 6, 7 / 9, 10, 11)에 대하여 회전된 커널 (9, 8, 7 / 6, 5, 4 / 3, 2, 1)과 원소별 곱셈 및 합산을 수행하였다. 각 곱셈은 1×9 , 2×8 , 3×7 , 5×6 , 6×5 , 7×4 , 9×3 , 10×2 , 11×1 순으로 이루어지며, 이를 모두 더하면 총합은 192가 된다.

이와 동일한 방식으로 우측 상단 출력 c12은 (2, 3, 4 / 6, 7, 8 / 10, 11, 12) 영역과의 MAC 연산을 통해 18, 24, 28, 36, 35, 32, 30, 22, 12의 곱셈 결과가 도출되며, 이들의 합은 237이다.

또한 하단 영역의 계산에서는 좌측 하단 c21이 (5, 6, 7 / 9, 10, 11 / 13, 14, 15)와 회전된 커널의 곱을 수행하며 각 항의 곱셈 결과는 45, 48, 49, 54, 50, 44, 39, 28, 15로 산출되고, 해당 값의 총합은 372로 계산된다. 동일하게 우측 하단 c22에 대해서는 (6, 7, 8 / 10, 11, 12 / 14, 15, 16)과 곱셈을 수행한 결과 54, 56, 56, 60, 55, 48, 42, 30, 16이 도출되며, 이들의 합산 결과는 417이다.

따라서 수학적 Convolution 정의에 따른 최종 출력 Feature Map은 (192, 237 / 372, 417)으로 얻어진다. 한편, FPGA 상의 실제 연산 과정에서는 Processing Element 내부의 accumulator가 8비트(0~255) 범위를 가지므로, 산출 값이 255를 초과할 경우 overflow가 발생한다. 그 결과 372는 255를 초과하여 $372 - 255 = 116$ 으로 변환되고, 동일하게 417은 $417 - 255 = 161$ 로 변환된다. 이로 인해 하드웨어 상에서 관측되는 실제 출력은 (192, 237 / 116, 161)이며, 이는 연산 오류가 아닌 accumulator 비트폭 제한에 따른 정상적인 overflow 동작임을 확인할 수 있었다.

7. Conclusion

본 프로젝트는 동일한 4×4 입력과 3×3 kernel을 대상으로 Single PE, 2×2 systolic array, 3×3 systolic array 세 가지 구조를 하나의 시스템 안에서 구현하고 비교함으로써, CNN 가속기에서의 병렬성·dataflow 전략·제어 복잡도가 실제 하드웨어 성능에 어떤 영향을 미치는지를 종합적으로 분석하는 데 목적이 있었다. 세 구조는 모두 동일한 convolution 결과(c11-c22)를 정확하게 출력하였으나, 구조적 특성과 데이터 이동 방식의 차이에 따라 연산 지연, 제어 오버헤드, 파이프라인 효율에서 뚜렷한 성능 차이가 나타났다.

Single PE 구조는 하나의 PE에서 모든 MAC 연산을 순차적으로 수행하는 baseline 형태로, 제어가 단순하고 하드웨어 자원이 가장 적게 소모된다는 장점이 있다. 그러나 직렬 처리 특성상 약 760 ns의 가장 긴 latency를

기록하였다. 내부적으로 단일 accumulator와 FSM 기반 입력 선택을 통해 c11~c22를 순차 계산하도록 구성되어 있어 구현과 검증이 용이했으나, 근본적으로 병렬성이 부재하기 때문에 cycle 소모가 크게 증가하였다.

2×2 systolic array는 네 개의 PE가 배열 형태로 배치되어 병렬성을 제공하지만, kernel 크기(3×3)와 array 크기(2×2)가 일치하지 않는 구조적 제약으로 인해 weight를 한 번에 고정할 수 없다. 이로 인해 연산은 세 단계의 weight-loading phase로 나뉘며, 각 phase마다 kernel의 특정 영역만 부분적으로 계산된다. 또한 convolution 특성상 생성되는 일부 partial sum중 최종 출력에 해당하는 값만 accumulator로 모아야 한다. 이러한 이유로 전체 FSM 제어기가 복잡해지고, 병렬 구조임에도 불구하고 weight 재배치 및 입력 정렬 과정에서 발생하는 오버헤드 때문에 latency는 약 580 ns로 개선 폭이 기대보다 제한적이었다.

3×3 systolic array는 kernel과 PE 배열이 1:1로 정확히 대응하는 구조로, Weight-Stationary 방식이 가장 효과적으로 적용될 수 있었다. 초기 weight를 단 한 번 로드한 뒤 고정된 상태에서, row-buffer 기반 input streaming을 통해 입력이 한 cycle씩 지연되며 자연스러운 파이프라인을 형성한다. 모든 PE가 연산에 참여하고 partial sum의 이동 경로도 명확하게 정렬되므로 데이터 재사용률이 최대화되고, 연산 자원의 활용도가 가장 높아진다. 그 결과 전체 연산은 약 360 ns로 세 구조 중 가장 짧은 latency를 보였으며, 병렬성이 효과적으로 활용될 때 얻을 수 있는 성능 향상을 잘 보여주었다.

세 구조를 종합적으로 비교했을 때 다음과 같은 핵심적 통찰을 얻을 수 있었다.

1. **PE 개수 증가가 성능 증가를 단순히 보장하지 않는다.**

Array 크기와 kernel 크기가 일치하지 않으면 weight 이동, partial sum 정렬, 입력 streaming 등에서 제어 오버헤드가 크게 증가하여 오히려 효율이 떨어질 수 있다.

2. **dataflow 선택(WS/OS/IS)은 구조 성능을 결정짓는 핵심 요소다.**

3×3 구조가 가장 높은 효율을 보인 이유는 WS 방식이 구조적으로 완벽히 들어맞았기 때문이다.

3. **local buffer, row-buffer, shift register 구성은 파이프라인 효율을 좌우한다.**

입력 지연 타이밍과 psum 정렬이 정확해야만 연산이 제대로 파이프라인화된다.

4. **controller FSM의 설계는 전체 CNN 시스템의 안정성을 결정한다.**

세 구조를 하나의 Top 시스템에서 순차적으로 실행하기 위해서는 enable/done 타이밍이 정확하게 맞아야 하고, 이는 실제 하드웨어에서 특히 중요하다.

5.

또한 본 프로젝트는 단순 연산 모듈 설계를 넘어, Memory-Computation-Display까지 연결된 완전한 end-to-end CNN 연산 흐름을 FPGA 상에서 구현했다는 점에서 의의가 크다. 특히 7-segment 기반 출력 구조는 RTL이 실제 보드 동작으로 이어지는 과정을 직접 경험하게 해주었으며, 이는 하드웨어 기반 CNN 가속기의 전체 동작 메커니즘을 이해하는 데 중요한 학습 경험이 되었다.

종합적으로 본 프로젝트는 병렬성, dataflow 구조, 제어 복잡도, 파이프라인 전략이 서로 어떻게 맞물려 전체 CNN 하드웨어 성능을 결정하는지를 실제 설계와 구현을 통해 깊이 이해할 수 있었던 작업이었다.

참고 문헌

- [1] Sze, V., Chen, Y.-H., Yang, T.-J., and Emer, J. S., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017. DOI: 10.1109/JPROC.2017.2761740.)